

DOKUMENTACJA PROJEKTU

PODSTAWY BAZ DANYCH

SKLEP SPOŻYWCZY | ORACLE

Spis treści

Diagramy	4
Notacja Chen'a	4
Notacja Barker'a	5
Notacja UML.....	6
Założenia i warunki poprawności modelu	7
1. Struktura Organizacyjna i Pracownicy.....	7
2. Proces Sprzedaży i Obsługi Klienta	7
3. Logistyka i Magazynowanie	7
4. Reklamacje i Serwis	7
Podsumowanie techniczne	8
Model relacyjny bazy danych.....	8
Diagram wynikowy	8
Opis konwersji z modelu E-R	8
Implementacja fizyczna modelu relacyjnego	9
Zwartość tabel	10
DOSTAWCY	10
KASJER	11
KATEGORIE.....	11
KIEROWNIK	11
KLIENCI.....	12
MAGAZYNIER.....	12
MAGAZYNY.....	13
POZYCJE_ZAMOWIENIA	14
PRACOWNICY	15
PRODUKTY	16
PRODUKTY_ALERGENY	16

REKLAMACJE	17
ZAMOWIENIA	17
Sekwencje zapytań (dopisywanie danych do bazy).....	18
Przyjęcie towaru (Dostawca → Produkt → Alergeny)	18
Kadry (Pracownik → Kasjer).....	18
Sprzedaż (Klient → Zamówienie → Pozycje)	19
Reklamacja (Powiązanie z istniejącą sprzedażą).....	19
Sekwencje zapytań (modyfikowanie i usuwanie danych w bazie)	19
Modyfikacje danych (UPDATE)	19
1. Awans pracownika (Zmiana roli i płacy)	19
2. Sezonowa zmiana cen dla konkretnego dostawcy.....	20
3. Reorganizacja magazynu	20
Usuwanie danych (DELETE).....	20
1. Pełne anulowanie zamówienia.....	20
2. Zakończenie współpracy z dostawcą	20
3. Rozwiązanie stosunku pracy.....	21
Sekwencje zapytań (pobieranie danych z bazy)	22
1. Zestawienie produktów z ich dostawcami.....	22
2. Zestawienie produktów razem z ich alergenami (wyświetl też te bez alergenów)	23
3. Oblicza ile produktów jest na kg a ile na szt	24
4. Kasjerzy odpowiedzialni za kasy 2 i 4	24
5. Suma wydatków każdego klienta (Tylko ci, co wydali ponad 50 zł).....	24
6. Średnia pensja na różnych stanowiskach (na podstawie przełożonego)	25
7. Szukanie produktów spożywczych zaczynających się od K lub M	25
8. Wybór zamówień opłaconych Blikiem lub Kartą	25
9. Porównanie cen z kategorią "Nabiał"	26
10. Pracownicy, którzy zarabiają więcej niż średnia w firmie	27
11. Produkty, które należą do kategorii "Nabiał"	28
12. Sprawdzenie, czy dany produkt był kiedykolwiek reklamowany	28
13. Najdroższe zamówienie ze wszystkich.....	29
14. Produkty, których cena jest wyższa niż średnia cena w ich własnej kategorii	29
15. Wyświetlenie listy alergenów dla produktów, które je posiadają:.....	30
Procedury składowane	30
zlicz_produkty_wedlug_ceny	30
przyznaj_premie.....	31
Wyzwalacze.....	33

trg_walidacja_ceny	33
trg_limit_podwyzki	33
trg_oznacz_promocja	34
Aplikacja kliencka.....	35
main_gui.py	35
db_operations.py	41
report_manager.py	45
Opis programu.....	48
Integracja raportowania z aplikacją.....	48
Interfejs (zakładkami).....	49
KATEGORIE	49
DOSTAWCY	49
KLIENCI	50
MAGAZYNY	50
PRODUKTY.....	51
RAPORTY	51

Diagramy

Notacja Chen'a

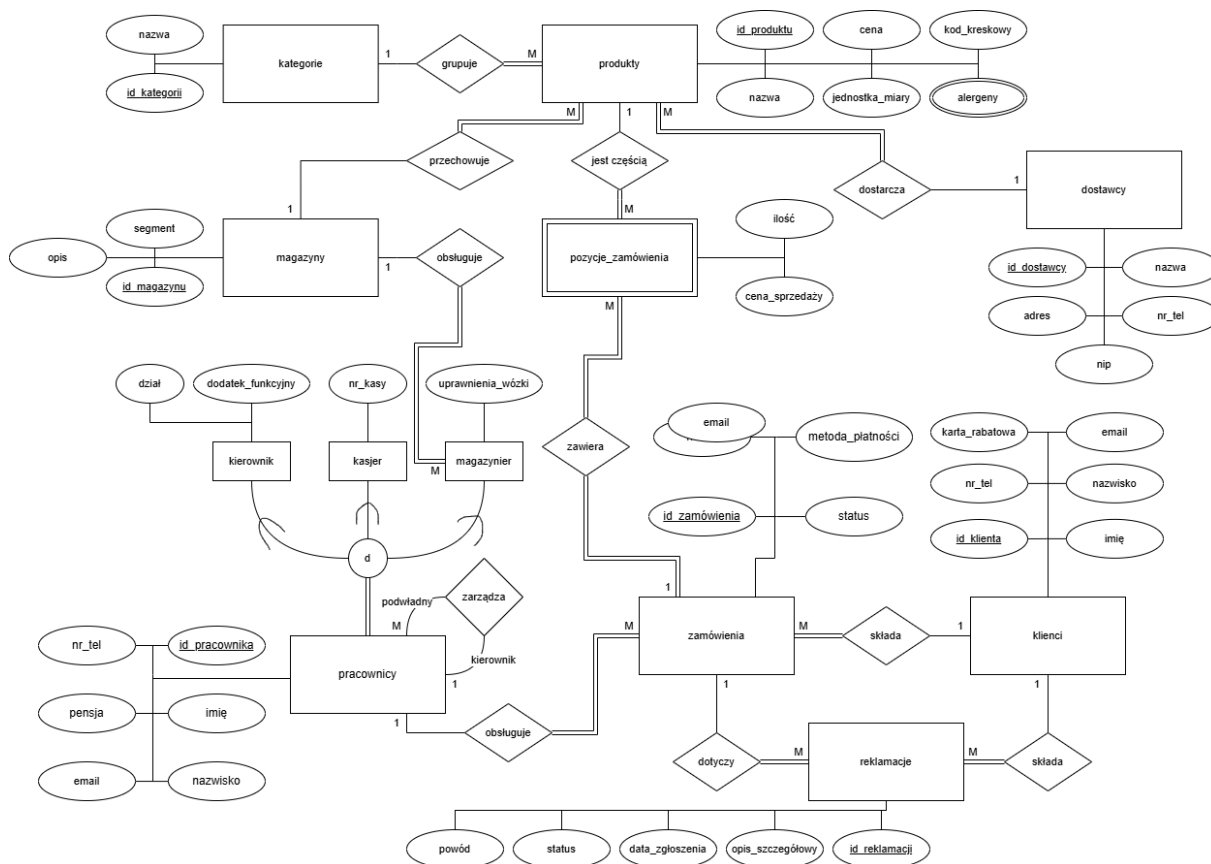


Diagram został wykonany w programie online **Draw.io (diagrams.net)**.

Notacja Barker'a

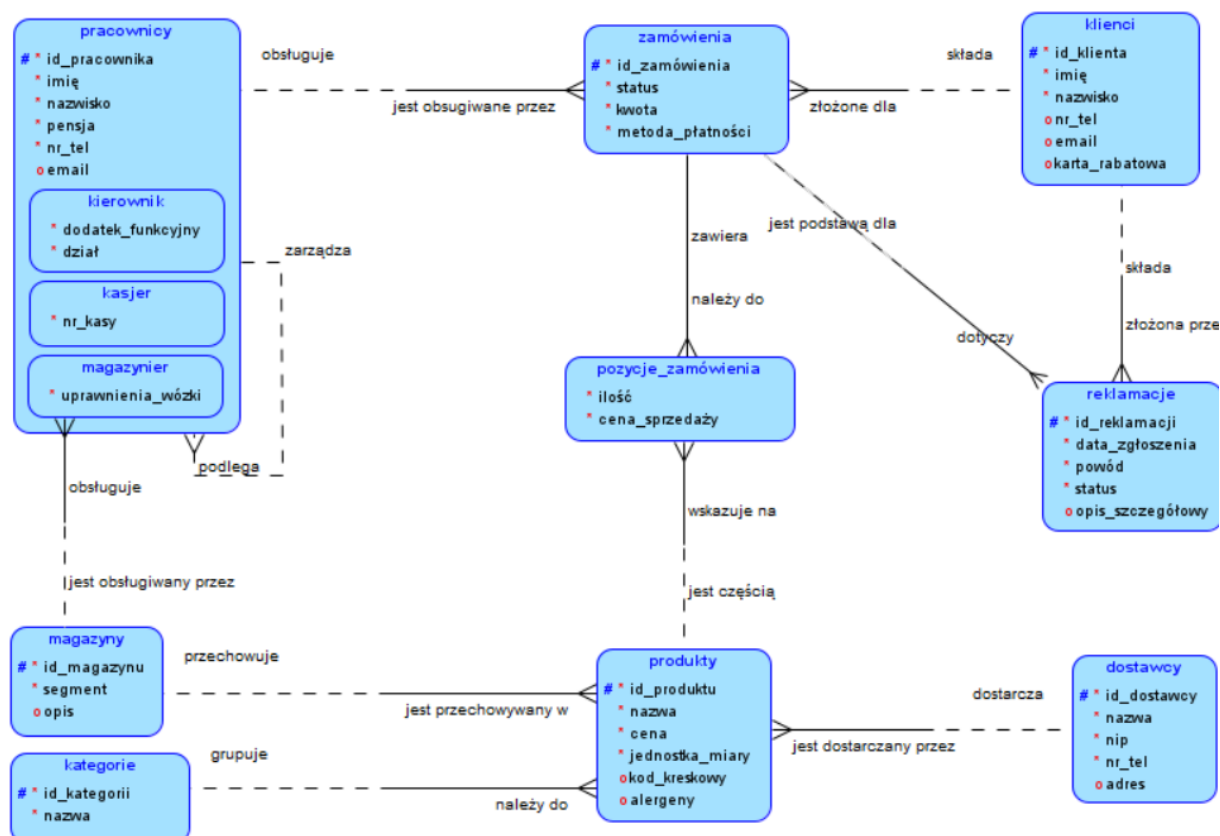


Diagram został wykonany w programie **Oracle SQL Developer Data Modeler**.

Notacja UML

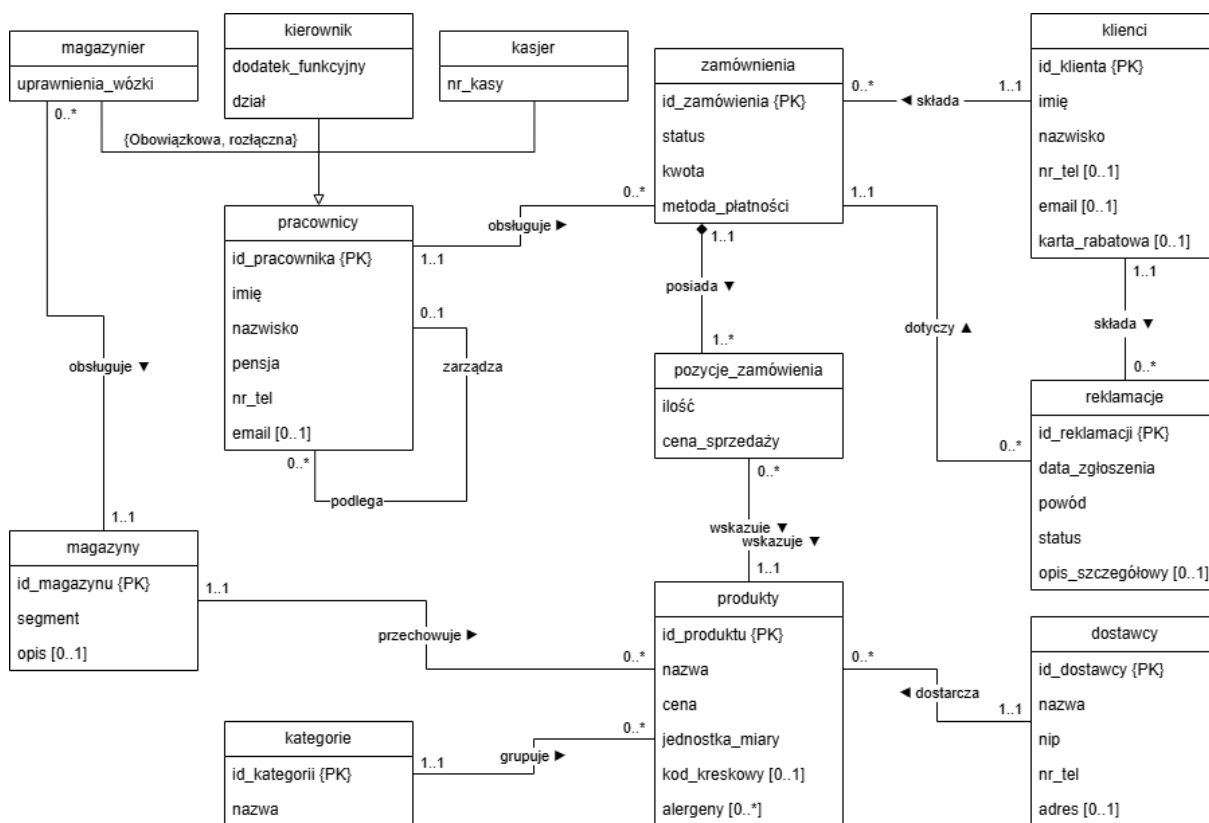


Diagram został wykonany w programie online **Draw.io (diagrams.net)**.

Założenia i warunki poprawności modelu

Poniższe zestawienie opisuje zasady, które muszą zostać spełnione w rzeczywistym systemie, aby struktura bazy danych była spójna z powyższymi diagramami ([Chen'a](#), [Barker'a](#), [UML](#)):

1. Struktura Organizacyjna i Pracownicy

- **Kompletność i rozłączność personelu:** Każdy pracownik w systemie musi posiadać jedną z trzech ról: Kierownik, Kasjer lub Magazynier (hierarchia {Obowiązkowa, rozłączna}). Nie jest możliwe istnienie "ogólnego" pracownika bez przypisanej specjalizacji.
- **Hierarchia służbowa:** System dopuszcza strukturę drzewiastą (relacja unarna zarządza / podlega). Przyjmuje się, że każdy podwładny ma maksymalnie jednego bezpośredniego przełożonego, natomiast kierownik najwyższego szczebla (np. dyrektor) nie posiada nad sobą nikogo (liczność 0..1).
- **Przypisanie do stanowisk:** Magazynier musi być na stałe przypisany do konkretnego magazynu (relacja 1..1), co pozwala na precyzyjne określenie odpowiedzialności za towar.

2. Proces Sprzedaży i Obsługi Klienta

- **Tożsamość klienta:** Zamówienia mogą składać wyłącznie klienci zarejestrowani w bazie danych. Każde zamówienie jest przypisane do dokładnie jednego klienta oraz jednego pracownika obsługującego transakcję (kasjera).
- **Integralność zamówienia:** Zamówienie nie może istnieć w systemie bez przynajmniej jednej pozycji towarowej (liczność 1..* przy pozycje_zamówienia).
- **Zasada kompozycji:** Pozycje zamówienia są nierozzerwalnie związane z nagłówkiem zamówienia (czarny romb). Usunięcie zamówienia z bazy danych skutkuje automatycznym usunięciem wszystkich jego pozycji (kaskadowość).
- **Historyczność cen:** Cena sprzedaży w tabeli pozycje_zamówienia jest ceną z momentu dokonania transakcji i może różnić się od aktualnej ceny katalogowej w tabeli produkty.

3. Logistyka i Magazynowanie

- **Kategoryzacja towarów:** Każdy produkt musi należeć do dokładnie jednej kategorii oraz być dostarczany przez dokładnie jednego dostawcę (uproszczenie modelu dla zachowania spójności relacji 1..1 z produktem).
- **Lokalizacja towaru:** Każdy produkt znajduje się w konkretnym magazynie. System nie przewiduje na tym etapie rozbicia jednego produktu na wiele różnych magazynów (relacja 1:M między magazynem a produktami).

4. Reklamacje i Serwis

- **Weryfikacja zakupowa:** Reklamacja może zostać złożona wyłącznie do zamówienia, które już istnieje w systemie.
- **Relacja trójstronna:** Reklamacja wiąże ze sobą klienta i konkretne zamówienie, co pozwala na automatyczną weryfikację uprawnień klienta do zgłoszenia roszczenia.

Podsumowanie techniczne

Model uznaje się za poprawny, gdy zapewniona jest integralność referencyjna (brak "osieroconych" rekordów, np. pozycji bez zamówienia) oraz gdy każdy atrybut oznaczony jako obowiązkowy posiada wartość w bazie danych.

Model relacyjny bazy danych

Diagram wynikowy

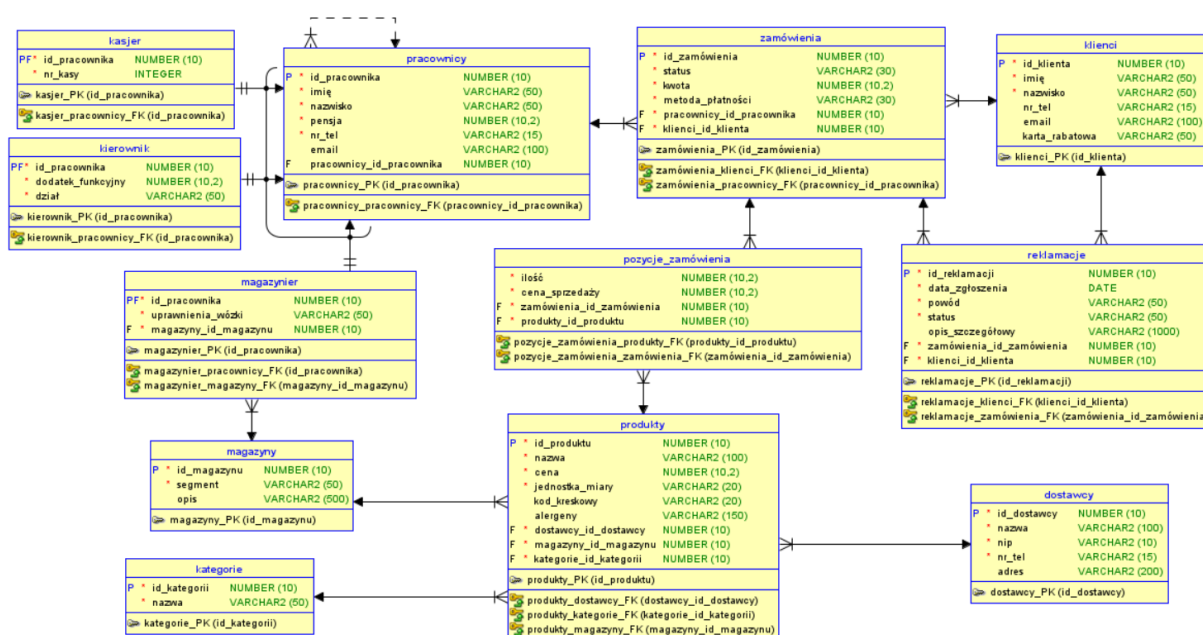


Diagram został wykonany automatycznie w programie **Oracle SQL Developer Data Modeler** na podstawie wcześniej już utworzonego diagramu w notacji Barker'a.

Opis konwersji z modelu E-R

1. Przekształcenie encji w tabele

- Każda **encja** zdefiniowana na modelu logicznym (np. produkty, klienci, zamówienia) została przekształcona w fizyczną **tabelę** w modelu relacyjnym.
- Nazwy encji stały się nazwami tabel.

2. Mapowanie atrybutów na kolumny i typy danych

- Atrybuty modelu **logicznego** stały się kolumnami tabel.
- Typy logiczne zostały zmapowane na fizyczne typy danych bazy **Oracle**:
 - NUMERIC** został zamieniony na **NUMBER**.
 - VARCHAR** został zamieniony na **VARCHAR2**.
 - DATE** pozostał typem **DATE**.
- Atrybuty oznaczone jako Mandatory (*) w modelu logicznym otrzymały w bazie status **NOT NULL**.

3. Migracja kluczy i realizacja relacji

Jest to najważniejszy element konwersji, który decyduje o spójności bazy:

- **Relacje 1:N (Jeden-do-Wielu):** Klucze główne z encji po stronie „jeden” (np. klienci) zostały przemieszczone (zmigrowane) jako klucze obce (FK) do tabel po stronie „wiele” (np. zamówienia).
- **Relacja M:N (Wiele-do-Wielu):** Relacja między produkty a zamówienia została rozwiązana poprzez utworzenie tabeli asocjacyjnej pozycje_zamówienia, która przejęła klucze główne z obu powiązanych tabel.
- **Relacja Unarna (Samozwrotna):** W tabeli pracownicy utworzono klucz obcy wskazujący na tę samą tabelę, co pozwala na odwzorowanie struktury podległości (pracownik-przełożony).

4. Obsługa hierarchii (Subtypów)

- Zastosowano strategię transformacji, w której każdy subtyp pracownika (kierownik, kasjer, magazynier) otrzymał własną tabelę.
- Tabele **subtypów** są powiązane z tabelą **nadtypu** (pracownicy) poprzez klucze PF (Primary-Foreign Key), co zapewnia unikalność identyfikatorów w całej strukturze kadrowej.

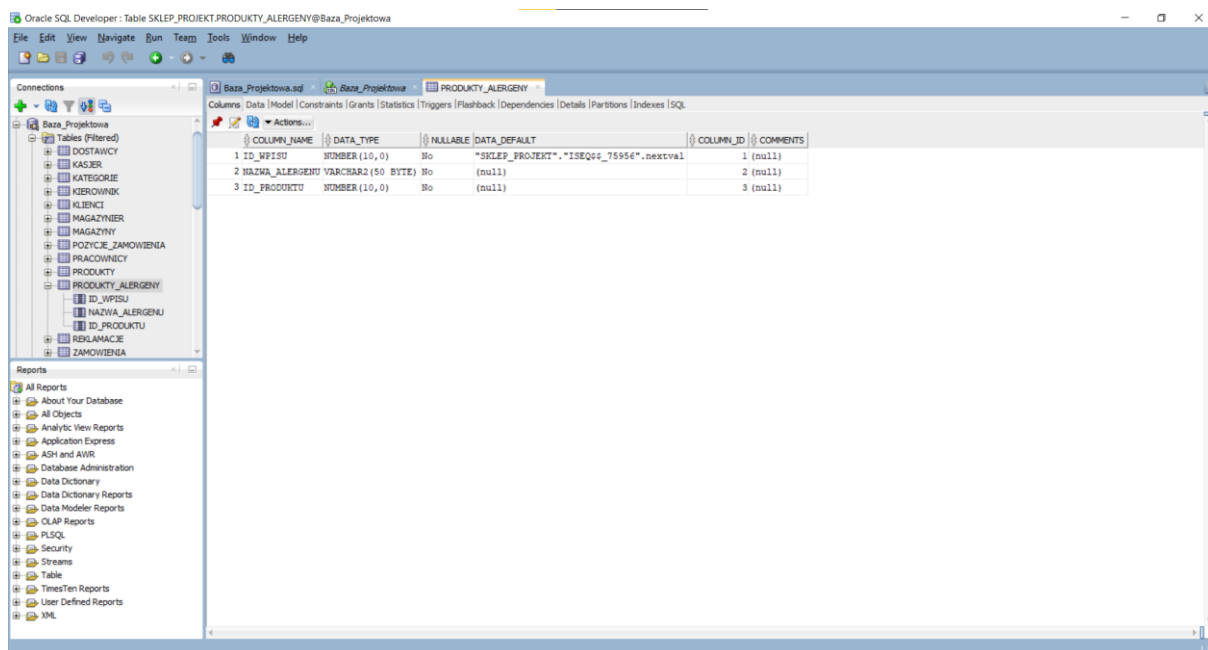
5. Definicja więzów spójności

- **Klucze Główne (P):** Unikalne identyfikatory UID z modelu logicznego stały się kluczami głównymi w modelu relacyjnym.
- **Klucze Obce (F):** Zostały automatycznie wygenerowane i nazwane (np. reklamacje_klienci_FK), aby zapewnić integralność referencyjną danych.

Implementacja fizyczna modelu relacyjnego

- **Cel:**
Przeniesienie logicznej struktury projektu do fizycznej instancji bazy danych Oracle XE.
- **Wykorzystane narzędzie:**
Środowisko Oracle SQL Developer oraz język SQL (skrypty DDL).
- **Wynik:**
Poprawne utworzenie 13 powiązanych ze sobą tabel w schemacie użytkownika sklep_projekt.

W celu zapewnienia **integralności danych**, tabele zostały utworzone w kolejności uwzględniającej zależności wynikające z kluczy obcych. Zastosowano mechanizm **IDENTITY** dla automatycznej numeracji rekordów, co ułatwi późniejszą obsługę bazy z poziomu aplikacji w języku Python. Pierwotny **atrybut wielowartościowy** alergeny (przechowywany jako ciąg tekstowy) został wydzielony do dedykowanej tabeli asocjacyjnej **produkty_alergeny**.



Widok z okna programu Oracle SQL Developer, utworzone tabele oraz widok tabeli **produkty_alergeny** (atrybut wielowartościowy).

Zwartość tabel

Uzupełniłem wszystkie 13 tabel w taki sposób, aby wszystkie dane były spójne i ukazywały pełną funkcjonalność bazy danych.

DOSTAWCY

ID_DOSTAWCY	NAZWA	NIP	NR_TEL	ADRES
1	1 Zakłady Mięsne Dobropol	5554443322	321115566	ul. Masarska 8, Katowice
2	2 Słodka Dolina	2223334455	613334455	(null)
3	3 Polmlek Sp. z o.o.	1234567890	221112233	ul. Mleczna 5, Warszawa
4	4 Hurtownia Warzyw Świeżość	1112223344	583337788	(null)
5	5 Piekarnia Chrupiący Róg	9876543210	223334455	ul. Żytnia 12, Kraków
6	6 Źródło Natury Sp. z o.o.	3334445566	712223344	ul. Zdrojowa 1, Wrocław
7	7 Pupil Food Sp. z o.o.	5556667788	413334455	(null)
8	8 Mroźny Świat S.A.	7776665544	125551122	ul. Lodowa 44, Łódź
9	9 Sady Mazowieckie	9998887766	254449900	ul. Owocowa 3, 05-600 Grójec
10	10 Miyn i Spółka	4445556677	174445566	ul. Pszenna 2, Rzeszów
11	11 Herbaciarnia Świata	6667778899	855556677	ul. Aromatyczna 9, Białystok
12	12 Przyprawy Orientu	3332221100	814445566	(null)
13	13 Czysty Dom Hurt	8889990011	521112233	ul. Chemiczna 30, Bydgoszcz
14	14 Techno-Sklep Hurtownia	7770001112	565556677	ul. Przemysłowa 50, 87-100 Toruń
15	15 Piękno i Zdrowie	1110002223	342223344	ul. Urody 4, Kalisz

Tabela przechowuje dane kontrahentów (hurtowni i producentów), od których sklep pozyskuje towar. Zawiera nazwy firm, ich numery NIP oraz dane kontaktowe niezbędne do sprawnej logistyki i rozliczeń.

KASJER

ID_PRACOWNIKA	NR_KASY
1	14
2	15
3	16
4	17
5	18
6	19
7	20
8	21
9	22
10	23

Rejestruje przypisanie konkretnych pracowników do stanowisk kasowych w systemie sprzedażowym. Pozwala na szybką identyfikację personelu operującego na danej kasie w celu rozliczania zmian i monitorowania sprzedaży.

KATEGORIE

ID_KATEGORII	NAZWA
1	1 Nabiał i jaja
2	2 Pieczywo i wyroby cukiernicze
3	3 Mięso, drób i wędliny
4	4 Warzywa świeże
5	5 Owoce świeże
6	6 Mrożonki i dania gotowe
7	7 Napoje, wody i soki
8	8 Słodkowie i słone przekąski
9	9 Produkty sypkie (mąki, kasze, makarony)
10	10 Kawy, herbaty i kakao
11	11 Chemia domowa i środki czystości
12	12 Kosmetyki i higiena osobista
13	13 Karma i akcesoria dla zwierząt
14	14 Przyprawy i dodatki do dań
15	15 Artykuły przemysłowe i sezonowe

Zawiera słownik grup asortymentowych, takich jak nabiał, pieczywo czy chemia domowa, ułatwiając katalogowanie produktów. Służy do logicznego porządkowania całej oferty sklepu oraz generowania przejrzystych raportów sprzedaży według działów.

KIEROWNIK

ID_PRACOWNIKA	DODATEK_FUNKCYJNY	DZIAL
1	1	3000 Zarząd
2	2	1500 Logistyka i Magazyn
3	3	1200 Sprzedaż i Kasy

Ewidencjonuje kadrę zarządzającą poszczególnymi działami sklepu (np. Zarząd, Logistyka) wraz z przysługującymi im dodatkami funkcyjnymi.

KLIENCI

ID_KLIENTA	IMIE	NAZWISKO	NR_TEL	EMAIL	KARTA_RABATOWA
1	Jan	Kowalski	601234567	j.kowalski@poczta.pl	KRT-10001
2	Anna	Nowak	784112233	a.nowak@poczta.pl	(null)
3	Piotr	Wiśniewski	512998776	p.wisniewski@gmail.com	KRT-10002
4	Maria	Wójcik	660554332	m.wojcik@wp.pl	(null)
5	Krzysztof	Lewandowski	(null)	k.lewandowski@poczta.pl	KRT-10003
6	Agnieszka	Zielińska	881223344	a.zielinska@gmail.com	KRT-10004
7	Tomasz	Szymański	505443221	t.szymanski@poczta.pl	(null)
8	Małgorzata	Woźniak	692887112	m.wozniak@wp.pl	KRT-10005
9	Paweł	Kozłowski	730111999	p.kozlowski@gmail.com	(null)
10	Barbara	Jankowska	531442553	b.jankowska@poczta.pl	KRT-10006
11	Marcin	Mazur	609887776	m.mazur@wp.pl	KRT-10007
12	Elżbieta	Krawczyk	888777666	e.krawczyk@gmail.com	(null)
13	Michał	Kaczmarek	501555666	m.kaczmarek@poczta.pl	KRT-10008
14	Zofia	Stępień	(null)	(null)	(null)
15	Adam	Grabowski	722333444	a.grabowski@gmail.com	KRT-10009
16	Jolanta	Kwiatkowska	604506708	(null)	(null)
17	Robert	Kamiński	(null)	r.kaminski@wp.pl	(null)

Gromadzi dane osobowe i kontaktowe osób dokonujących zakupów, umożliwiając ich identyfikację w systemie.

MAGAZYNIER

ID_PRACOWNIKA	UPRAWNIENIA_WOZKI	MAGAZYNY_ID_MAGAZYNU
1	4 Wózki widłowe	1
2	5 Podstawowe	2
3	6 Wózki widłowe	3
4	7 Podstawowe	4
5	8 Wózki widłowe	5
6	9 Podstawowe	6
7	10 Wózki widłowe	7
8	11 Podstawowe	8
9	12 Wózki widłowe	9
10	13 Podstawowe	10
11	24 Podstawowe	11
12	25 Podstawowe	12
13	26 Wózki widłowe	13
14	27 Podstawowe	14
15	28 Podstawowe	15
16	29 Wózki widłowe	2
17	30 Podstawowe	15

Przechowuje informacje o pracownikach magazynu, ich uprawnieniach do obsługi wózków widłowych oraz przypisaniu do konkretnych stref składowania. Umożliwia sprawne zarządzanie personelem logistycznym i monitorowanie kompetencji kadry magazynowej.

MAGAZYNY

ID_MAGA...	SEGMENT	OPIS
1	1 A-01	Chłodnia: produkty nabiałowe i sery
2	2 A-02	Chłodnia: mięso, wędliny i drób
3	3 B-01	Magazyn suchy: kasze, maki, makarony
4	4 B-02	Magazyn suchy: słodyczne i przekąski
5	5 C-01	Strefa napojów: wody i soki
6	6 C-02	Strefa napojów: napoje gazowane i energetyki
7	7 D-01	Dział warzywny: warzywa korzeniowe
8	8 D-02	Dział owocowy: owoce egzotyczne
9	9 E-01	Mroźnie: lody i owoce mrożone
10	10 E-02	Mroźnie: gotowe dania i warzywa mrożone
11	11 F-01	Chemia domowa: środki czystości
12	12 F-02	Kosmetyki i higiena osobista
13	13 G-01	Karma i akcesoria dla zwierząt
14	14 H-01	Artykuły przemysłowe i sezonowe
15	15 I-01	Strefa zwrotów i reklamacji

Definiuje fizyczne strefy i segmenty składowania towarów.

POZYCJE_ZAMOWIENIA

	ID_POZYCJI	ILOSC	CENA_SPRZEDAZY	ZAMOWIENIA_ID_ZAMOWI...	PRODUKTY_ID_PRODUKTU
1	37	1	4,2	1	3
2	36	2	3,49	1	1
3	38	1	15,99	2	15
4	39	10	0,85	3	17
5	40	1	2,8	3	16
6	42	1	5,8	4	22
7	41	2	12,5	4	21
8	43	0,5	14,99	4	4
9	44	1	4,8	4	8
10	45	1	18,9	5	18
11	46	2	3,5	6	19
12	47	1	3,1	6	9
13	48	1	42,5	6	2
14	49	2	18,5	7	6
15	50	6	2,2	8	7
16	51	1	4,99	8	24
17	56	2	12,5	9	12
18	55	1	45	9	10
19	54	1	14,5	9	25
20	53	1	8,9	9	11
21	52	1	34	9	26
22	57	3	3,49	9	1
23	58	1	42,5	9	2
24	59	1	3,8	10	5
25	60	1	14,5	11	25
26	61	4	3,2	11	23
27	62	1	45	12	10
28	63	1	2,1	13	29
29	65	2	3,49	13	1
30	64	1	4,2	13	3
31	66	0,5	42,5	14	2
32	67	3	6,99	15	20
33	68	1	3,8	16	5
34	69	2	2,8	16	16
35	70	1	12,5	16	21
36	71	1	12,5	17	12
37	72	2	2,1	17	29
38	73	1	15,99	18	15
39	74	1	14,5	19	25
40	75	2	3,8	19	5
41	76	3	1,5	20	14
42	77	1	3,1	20	9
43	78	4	3,49	21	1
44	79	2	2,8	21	16

Rejestruje szczegółowy wykaz produktów zakupionych w ramach każdej transakcji wraz z ich ilością i ceną sprzedaży. Stanowi kluczowy łącznik między zamówieniem a produktami, pozwalając na precyzyjne rozliczenie każdego koszyka zakupowego.

PRACOWNICY

ID_PRACOWNIKA	IMIE	NAZWISKO	PENSJA	NR_TEL	EMAIL	PRACOWNICY_ID_PRACOWNIKA
1	1 Grzegorz	Rowak	11200	600180180	g.rowak@sklep.pl	(null)
2	2 Paweł	Kowalski	7500	530730100	p.kowalski@sklep.pl	1
3	3 Krystyna	Ramecka	8000	640790202	k.ramecka@sklep.pl	1
4	4 Marek	Wiśniewski	4350,5	700111222	m.wisniewski@sklep.pl	2
5	5 Jan	Zieliński	4100	700222333	j.zielinski@sklep.pl	2
6	6 Filip	Nowakowski	4600,75	700333444	f.nowakowski@sklep.pl	2
7	7 Jacek	Kwiatkowski	4420	700444555	j.kwiatkowski@sklep.pl	2
8	8 Karol	Woźniak	4250,25	700555666	k.wozniak@sklep.pl	2
9	9 Robert	Kamiński	4580	700666777	r.kaminski@sklep.pl	2
10	10 Artur	Lewandowski	4150,5	700777888	a.lewandowski@sklep.pl	2
11	11 Szymon	Dąbrowski	4390	700888999	s.dabrowski@sklep.pl	2
12	12 Jakub	Kozłowski	4470,75	700999000	j.kozlowski@sklep.pl	2
13	13 Tomasz	Jankowski	4215	700000111	t.jankowski@sklep.pl	2
14	14 Anna	Bąk	4050	600111222	a.bak@sklep.pl	3
15	15 Ewa	Mazur	3980,5	600222333	e.mazur@sklep.pl	3
16	16 Olga	Kaczmarek	4120	600333444	o.kaczmarek@sklep.pl	3
17	17 Iga	Dudek	4000,75	600444555	i.dudek@sklep.pl	3
18	18 Maja	Zajac	4250	600555666	m.zajac@sklep.pl	3
19	19 Sara	Król	4180,25	600666777	s.krol@sklep.pl	3
20	20 Nina	Wieczorek	4090	600777888	n.wieczorek@sklep.pl	3
21	21 Mariusz	Pudzianowski	4250	600888001	m.pudzianowski@sklep.pl	3
22	22 Dariusz	Stolarski	4100,5	600888002	d.stolarski@sklep.pl	3
23	23 Sebastian	Kołodziej	4320	600888003	s.kolodziej@sklep.pl	3
24	24 Marta	Włodarczyk	4450	700900101	m.wlodarczyk@sklep.pl	2
25	25 Sylwia	Górska	4300,5	700900102	s.gorska@sklep.pl	2
26	26 Katarzyna	Wójcik	4650	700900103	k.wojcik@sklep.pl	2
27	27 Monika	Lis	4200,75	700900104	m.lis@sklep.pl	2
28	28 Barbara	Szymczak	4550	700900105	b.szymczak@sklep.pl	2
29	29 Jolanta	Kaczmarczyk	4380,25	700900106	j.kaczmarczyk@sklep.pl	2
30	30 Agnieszka	Krawczyk	4410	700900107	a.krawczyk@sklep.pl	2

Centralny rejestr personelu zawierający dane osobowe, pensje zasadnicze, numery telefonów oraz strukturę podległości służbowej. Jest podstawą do zarządzania kadrami, naliczania wynagrodzeń oraz dalszego przypisywania pracowników do konkretnych ról operacyjnych.

PRODUKTY

ID_PRODUKTU	NAZWA	CENA	JEDNOSTKA_MIARY	KOD_KRESKOWY	DOSTAWCY_ID_DOSTAWCY	MAGAZYNY_ID_MAGAZYN	KATEGORIE_ID_KATEGORII
1	1 Mleko 2% UHT	3,49	szt	5901234567890	3	1	1
2	2 Sznoka Konserwowa	42,5	kg	2100550000001	1	2	3
3	3 Chleb Żytni	4,2	szt	(null)	5	3	2
4	4 Pomidory Malinowe	14,99	kg	(null)	4	7	4
5	5 Jabłka Ligol	3,8	kg	(null)	9	8	5
6	6 Lody Waniłowe 1L	18,5	szt	5904433221100	8	9	6
7	7 Woda Gazowana 1.5L	2,2	szt	5905566778899	6	5	7
8	8 Czekolada Mleczna	4,8	szt	7622210001234	2	4	8
9	9 Mąka Pszena 1kg	3,1	szt	5908877665544	10	3	9
10	10 Kawa Ziarnista 500g	45	szt	4001234567890	11	6	10
11	11 Płyn do Naczyń	8,9	szt	5412345678901	13	11	11
12	12 Krem do rąk	12,5	szt	4005800123456	15	12	12
13	13 Karma dla psa 5kg	35	szt	5903322114455	7	13	13
14	14 Pieprz Czarny	1,5	szt	5900000001112	12	14	14
15	15 Baterie AA (4szt)	15,99	szt	4008400123456	14	14	15
16	16 Jogurt Naturalny 400g	2,8	szt	5901122334455	3	1	1
17	17 Bułka Kajzerka	0,85	szt	(null)	5	3	2
18	18 Kurczak Cały	18,9	kg	2100880000002	1	2	3
19	19 Marchew Polska	3,5	kg	(null)	4	7	4
20	20 Banany Premium	6,99	kg	(null)	9	8	5
21	21 Pizza Mrożona 4 Sery	12,5	szt	5909988776655	8	10	6
22	22 Sok Pomarańczowy 1L	5,8	szt	5907766554433	6	5	7
23	23 Herbatniki Maślane	3,2	szt	5904455667788	2	4	8
24	24 Ryż Biały 1kg	4,99	szt	5901212121212	10	3	9
25	25 Herbata Czarna 100t	14,5	szt	5903232323232	11	6	10
26	26 Proszek do prania 3kg	34	szt	5401122334455	13	11	11
27	27 Szampon do włosów	15,2	szt	4001122334455	15	12	12
28	28 Przynęta dla kota	4,5	szt	5905544332211	7	13	13
29	29 Sól Morska 1kg	2,1	szt	5909900990099	12	14	14
30	30 Żarówka LED E27	9,99	szt	4012345678901	14	14	15

Główny katalog towarów oferowanych przez sklep, zawierający ceny, jednostki miary, kody kreskowe oraz powiązania z dostawcami. Łączy każdy artykuł z jego kategorią oraz precyzyjnym miejscem składowania w magazynie.

PRODUKTY_ALERGENY

ID_WPISU	NAZWA_ALERGENU	ID_PRODUKTU
1	21 Laktoza	1
2	22 Laktoza	6
3	23 Jaja	6
4	24 Laktoza	16
5	25 Gluten	3
6	26 Gluten	9
7	27 Gluten	17
8	28 Gluten	21
9	29 Laktoza	21
10	30 Gluten	23
11	31 Laktoza	23
12	32 Jaja	23
13	33 Białko sojowe	2
14	34 Gorczyca	2
15	35 Gorczyca	18
16	36 Orzechy laskowe	8
17	37 Laktoza	8
18	38 Lecytyna sojowa	8
19	39 Zboża	13
20	40 Ryby	28

Tabela zawiera wykaz substancji uczulających przypisanych do konkretnych artykułów spożywczych w celu zapewnienia bezpieczeństwa konsumentom. Pozwala na przypisanie wielu alergenów (np. laktoza, gluten, jaja) do jednego produktu przy wykorzystaniu jego unikalnego identyfikatora.

REKLAMACJE

ID_REKLAMACJI	DATA_ZGLOSZENIA	POWOD	STATUS	OPIS_SZCZEGOLOWY	ZAMOWIENIA_ID_ZAMOWIENIA	KLIENCI_ID_KLIENTA
1	26/01/05	Uszkodzony towar	Uznana	Pęknięta obudowa proszku do prania (ID: 26).	9	8
2	27/01/07	Brak towaru	Uznana	(null)	9	8
3	28/01/10	Błąd w cenie	Zakończona	Niezgodność ceny płynu do naczyń (ID: 11).	9	8
4	29/01/02	Uszkodzony towar	Uznana	Rozlane mleko 2% (ID: 1).	1	1
5	30/01/04	Jakość produktu	Odrzucona	(null)	1	1
6	31/01/14	Uszkodzony towar	W rozpatrywaniu	Pęknięta torebka soli morskiej (ID: 29).	17	13
7	32/01/11	Błąd w cenie	Zakończona	Błąd przy naliczaniu herbatników maślanych (ID: 23).	11	10
8	33/01/27	Uszkodzony towar	Uznana	Zgniecione banany premium (ID: 20).	15	16
9	34/01/03	Uszkodzony towar	Nowa	Baterie AA wylane w opakowaniu (ID: 15).	18	15
10	35/01/30	Jakość produktu	Odrzucona	(null)	6	5
11	36/01/15	Inny powód	Nowa	Herbata czarna (ID: 25) - pomyłka przy wydaniu towaru.	19	17
12	37/01/12	Błąd w cenie	W rozpatrywaniu	(null)	21	2
13	38/01/08	Uszkodzony towar	Uznana	Pudełko z pizzą (ID: 21) mocno uszkodzone.	4	16
14	39/01/20	Błąd w cenie	Zakończona	Niezgodność ceny za Chleb Żytni (ID: 3).	13	12
15	40/01/24	Brak towaru	Odrzucona	Klient twierdzi, że brakowało maki (ID: 9).	20	1

Służy do ewidencjonowania zgłoszeń klientów dotyczących wadliwych towarów, błędów w cenie lub braków w dostawie. Gromadzi informacje o powodzie reklamacji, jej bieżącym statusie oraz szczegółach wady, wiążąc zgłoszenie z konkretnym zamówieniem i nabywcą.

ZAMOWIENIA

ID_ZAMOWIENIA	STATUS	KWOTA	METODA_PŁATNOSCI	PRACOWNICY_ID_PRACOWNIKA	KLIENCI_ID_KLIENTA
1	1 Zakończona	11,18	Karta	14	1
2	2 Zakończona	15,99	Blik	23	1
3	3 Zakończona	11,3	Gotówka	15	2
4	4 W trakcie realizacji	43,1	Blik	16	16
5	5 Zakończona	18,9	Karta	17	4
6	6 Zakończona	52,6	Gotówka	18	5
7	7 Anulowane	37	Karta	19	6
8	8 Zakończona	18,19	Blik	20	7
9	9 Zakończona	180,37	Karta	21	8
10	10 Zakończona	3,8	Gotówka	22	8
11	11 W trakcie realizacji	27,3	Blik	23	10
12	12 Zakończona	45	Karta	14	11
13	13 Zakończona	13,28	Gotówka	15	12
14	14 Zakończona	21,25	Blik	16	14
15	15 Anulowane	20,97	Karta	17	16
16	16 Zakończona	21,9	Gotówka	18	17
17	17 Zakończona	16,7	Karta	14	13
18	18 Zakończona	15,99	Gotówka	15	15
19	19 W trakcie realizacji	22,1	Blik	16	17
20	20 Zakończona	7,6	Blik	23	1
21	21 Zakończona	19,56	Karta	20	2

Rejestruje nagłówki każdej transakcji, przechowując informacje o statusie realizacji, łącznej kwocie do zapłaty oraz wybranej metodzie płatności. Wiąże proces zakupu z konkretnym klientem oraz pracownikiem odpowiedzialnym za sfinalizowanie sprzedaży.

Sekwencje zapytań (dopisywanie danych do bazy)

Przyjęcie towaru (Dostawca → Produkt → Alergeny)

Ten proces pokazuje, jak wprowadzić nowy asortyment do bazy, dbając o relacje z kontrahentem i bezpieczeństwo żywności.

```
-- 1. Dodanie nowego dostawcy
INSERT INTO dostawcy (id_dostawcy, nazwa, nip, nr_tel, adres)
VALUES (DEFAULT, 'Nazwa Firmy', 'NIP_NUMER', 'TELEFON', 'ADRES_SIEDZIBY');

-- 2. Dodanie produktu przypisanego do tego dostawcy
INSERT INTO produkty (id_produktu, nazwa, cena, jednostka_miary, kod_kreskowy,
dostawcy_id_dostawcy, magazyny_id_magazynu, kategorie_id_kategorii)
VALUES (DEFAULT, 'Nazwa Produktu', 9.99, 'szt/kg', 'KOD_EAN',
        {ID_DOSTAWCY_Z_KROKU_1},
        {ID_WYBRANEGO_MAGAZYNU},
        {ID_WYBRANEJ_KATEGORII});

-- 3. Przypisanie alergenu do nowego produktu
INSERT INTO produkty_alergeny (id_wpisu, nazwa_alergenu, id_produktu)
VALUES (DEFAULT, 'Nazwa Alergenu', {ID_PRODUKTU_Z_KROKU_2});
```

Kadry (Pracownik → Kasjer)

Pokazuje, jak ogólny rekord pracownika zyskuje konkretne uprawnienia w sklepie.

```
-- KROK 1: Rejestracja pracownika w ogólnym systemie kadrowym
-- To jest baza dla każdej innej roli; tutaj powstaje unikalny identyfikator
osoby.
INSERT INTO pracownicy (id_pracownika, imie, nazwisko, pensja, nr_tel, email,
pracownicy_id_pracownika)
VALUES (DEFAULT, 'Imię', 'Nazwisko', 4000, 'TELEFON', 'EMAIL',
        {ID_PRZEŁOŻONEGO_LUB_NULL});

-- KROK 2: Wybór i przypisanie roli (zależnie od stanowiska):
-- OPCJA A: Jeśli pracownik zostaje KASJEREM
INSERT INTO kasjer (id_pracownika, nr_kasy)
VALUES ({ID_PRACOWNIKA_Z_KROKU_1}, {NUMER_PRZYPISANEJ_KASY});

-- OPCJA B: Jeśli pracownik zostaje MAGAZYNIEREM
INSERT INTO magazynier (id_pracownika, uprawnienia_wozki,
magazyny_id_magazynu)
VALUES ({ID_PRACOWNIKA_Z_KROKU_1}, '{TYP_UPRAWNIENI}',
        {ID_PRZYPISANEGO_MAGAZYNU});

-- OPCJA C: Jeśli pracownik zostaje KIEROWNIKIEM
INSERT INTO kierownik (id_pracownika, dodatek_funkcyjny, dzial)
VALUES ({ID_PRACOWNIKA_Z_KROKU_1}, {KWOTA_DODATKU}, '{NAZWA_DZIAŁU}');
```

Sprzedaż (Klient → Zamówienie → Pozycje)

Łączy klienta z obsługą oraz konkretną listą zakupów.

```
-- 1. Rejestracja nowego klienta
INSERT INTO klienci (id_klienta, imie, nazwisko, nr_tel, email,
karta_rabatowa)
VALUES (DEFAULT, 'Imię', 'Nazwisko', 'TELEFON', 'EMAIL', 'NUMER_KARTY');

-- 2. Utworzenie nagłówka nowego zamówienia
INSERT INTO zamowienia (id_zamowienia, status, kwota, metoda_platnosci,
pracownicy_id_pracownika, klienci_id_klienta)
VALUES (DEFAULT, 'Nowe', 0, 'Metoda_Płatności', {ID_KASJERA_OBSŁUGUJĄCEGO},
{ID_KLIENTA_KUPUJĄCEGO});

-- 3. Dodanie towaru do koszyka (powtarzane dla każdego produktu)
INSERT INTO pozycje_zamowienia (id_pozycji, ilosc, cena_sprzedazy,
zamowienia_id_zamowienia, produkty_id_produktu)
VALUES (DEFAULT, {ILOŚĆ}, {CENA_W_CHWILI_ZAKUPU}, {ID_ZAMOWIENIA_Z_KROKU_2},
{ID_SKANOWANEGO_PRODUKTU});
```

Reklamacja (Powiązanie z istniejącą sprzedażą)

Wymaga istnienia wcześniejszej transakcji, aby zgłoszenie było ważne.

```
-- 1. Rejestracja zgłoszenia reklamacyjnego
INSERT INTO reklamacje (id_reklamacji, data_zgloszenia, powod, status,
opis_szczegolowy, zamowienia_id_zamowienia, klienci_id_klienta)
VALUES (DEFAULT, CURRENT_DATE, 'Powód_Reklamacji', 'Status', 'Opis_Wady',
{ID_REKLAMOWANEGO_ZAMÓWIENIA},
{ID_KLIENTA_KTÓRY_DOKONAŁ_ZAKUPU});
```

Sekwencje zapytań (modyfikowanie i usuwanie danych w bazie)

Modyfikacje danych (UPDATE)

Modyfikacje pozwalają na aktualizację stanów, cen oraz statusów procesów biznesowych bez konieczności usuwania rekordów.

1. Awans pracownika (Zmiana roli i płacy)

Scenariusz: Kasjer zostaje awansowany na Kierownika działu. Wymaga to aktualizacji pensji oraz dopisania/aktualizacji rekordu w tabeli funkcyjnej.

```
-- Krok 1: Podwyżka pensji zasadniczej
UPDATE pracownicy
SET pensja = {NOWA_WYŻSZA_KWOTA}
```

```
WHERE id_pracownika = {ID_AWANSOWANEGO_PRACOWNIKA};

-- Krok 2: Nadanie roli kierownika i przypisanie dodatku funkcyjnego
INSERT INTO kierownik (id_pracownika, dodatek_funkcyjny, dzial)
VALUES ({ID_AWANSOWANEGO_PRACOWNIKA}, {KWOTA_DODATKU}, '{NAZWA_DZIAŁU}');
```

2. Sezonowa zmiana cen dla konkretnego dostawcy

Scenariusz: Dostawca (np. "Polmlek") podnosi ceny wszystkich swoich produktów o 5%.

```
UPDATE produkty
SET cena = cena * 1.05
WHERE dostawcy_id_dostawcy = {ID_DOSTAWCY_KTÓRY_PODNIÓSŁ_CENY};
```

3. Reorganizacja magazynu

Scenariusz: Przeniesienie wszystkich produktów z jednej kategorii (np. Napoje) do innej strefy magazynowej.

```
UPDATE produkty
SET magazyny_id_magazynu = {ID_NOWEGO_MAGAZYNU_NP_CHŁODNI}
WHERE kategorie_id_kategorii = {ID_KATEGORII_NAPOJE};
```

Usuwanie danych (DELETE)

Usuwanie jest operacją **krytyczną**, która w Twojej bazie musi uwzględniać więzy integralności (nie można usunąć produktu, który jest w zamówieniu).

1. Pełne anulowanie zamówienia

Scenariusz: Klient rezygnuje z zakupów. Nie możemy usunąć zamówienia, dopóki w bazie istnieją jego pozycje.

```
-- Krok 1: Usunięcie wszystkich produktów przypisanych do tego zamówienia
DELETE FROM pozycje_zamówienia
WHERE zamowienia_id_zamowienia = {ID_ANULOWANEGO_ZAMÓWIENIA};

-- Krok 2: Usunięcie nagłówka zamówienia
DELETE FROM zamowienia
WHERE id_zamowienia = {ID_ANULOWANEGO_ZAMÓWIENIA};
```

2. Zakończenie współpracy z dostawcą

Scenariusz: Sklep przestaje zamawiać towary od danego dostawcy. Musimy usunąć jego produkty i alergeny przed usunięciem samej firmy.

```
-- Krok 1: Usunięcie alergenów powiązanych z produktami tego dostawcy
DELETE FROM produkty_alergeny
```

```
WHERE id_produktu IN (SELECT id_produktu FROM produkty WHERE
dostawcy_id_dostawcy = {ID_DOSTAWCY});
```

```
-- Krok 2: Usunięcie produktów tego dostawcy
```

```
DELETE FROM produkty
```

```
WHERE dostawcy_id_dostawcy = {ID_DOSTAWCY};
```

```
-- Krok 3: Usunięcie samego dostawcy
```

```
DELETE FROM dostawcy
```

```
WHERE id_dostawcy = {ID_DOSTAWCY};
```

3. Rozwiązanie stosunku pracy

Scenariusz: Pracownik odchodzi z pracy. Należy go usunąć z tabel funkcyjnych przed tabelą główną.

Dla kasjera:

```
-- Krok 1: Usunięcie przypisania do kasy
```

```
DELETE FROM kasjer
```

```
WHERE id_pracownika = {ID_ODCHODZĄCEGO_PRACOWNIKA};
```

```
-- Krok 2: Usunięcie rekordu z głównej tabeli pracowników
```

```
DELETE FROM pracownicy
```

```
WHERE id_pracownika = {ID_ODCHODZĄCEGO_PRACOWNIKA};
```

Dla magazyniera:

```
-- Krok 1: Usunięcie przypisania pracownika do magazynu i informacji o
uprawnieniach na wózki
```

```
DELETE FROM magazynier
```

```
WHERE id_pracownika = {ID_ODCHODZĄCEGO_MAGAZYNIERA};
```

```
-- Krok 2: Usunięcie pracownika z ogólnego rejestru kadr i listy płac
```

```
DELETE FROM pracownicy
```

```
WHERE id_pracownika = {ID_ODCHODZĄCEGO_MAGAZYNIERA};
```

Dla kierownika:

```
-- Krok 1: Obsługa podwładnych (Update)
```

```
-- Zanim usuniemy szefa, musimy przypisać jego pracowników do kogoś innego lub
ustawić im brak przełożonego (NULL).
```

```
UPDATE pracownicy
```

```
SET pracownicy_id_pracownika = {ID_NOWEGO_SZEFA_LUB_NULL}
```

```
WHERE pracownicy_id_pracownika = {ID_USUWANEGO_KIEROWNIKA};
```

```
-- Krok 2: Usunięcie wpisu z tabeli funkcyjnej
```

```
-- Czyścimy informację o tym, że ta osoba pełniła funkcję kierowniczą.
```

```
DELETE FROM kierownik
```

```
WHERE id_pracownika = {ID_USUWANEGO_KIEROWNIKA};

-- Krok 3: Usunięcie pracownika z głównej tabeli
-- Dopiero teraz, gdy nikt w bazie nie "patrzy" na to ID, możemy bezpiecznie
usunąć rekord.
DELETE FROM pracownicy
WHERE id_pracownika = {ID_USUWANEGO_KIEROWNIKA};
```

Sekwencje zapytań (pobieranie danych z bazy)

Poniższe zapytania zostały wykonane na bazie przez interfejs Visual Studio Code (ponieważ, był dla mnie wygodniejszy niż oryginalny od Oracle).

1. Zestawienie produktów z ich dostawcami

```
SELECT
    p.nazwa as nazwa_produktu,
    p.cena as cena_produktu,
    d.nazwa as nazwa_dost
FROM produkty p
INNER JOIN dostawcy d ON d.id_dostawcy = p.dostawcy_id_dostawcy;
```

	NAZWA_PRODUKTU	CENA_PRODUKTU	NAZWA_DOST
1	Kurczak Cały	18.9	Zakłady Mięsne Dobropol
2	Szynka Konserwowa	42.5	Zakłady Mięsne Dobropol
3	Herbatniki Maślane	3.2	Słodka Dolina
4	Czekolada Mleczna	4.8	Słodka Dolina
5	Mleko 2% UHT	3.49	Polmlek Sp. z o.o.
6	Jogurt Naturalny 400g	2.8	Polmlek Sp. z o.o.
7	Pomidory Malinowe	14.99	Hurtownia Warzyw Świeżość
8	Marchew Polska	3.5	Hurtownia Warzyw Świeżość
9	Chleb Żytni	4.2	Piekarnia Chrupiący Róg
10	Bułka Kajzerka	0.85	Piekarnia Chrupiący Róg
11	Sok Pomarańczowy 1L	5.8	Źródło Natury Sp. z o.o.
12	Woda Gazowana 1.5L	2.2	Źródło Natury Sp. z o.o.
13	Karma dla psa 5kg	35	Pupil Food Sp. z o.o.
14	Przysmak dla kota	4.5	Pupil Food Sp. z o.o.
15	Lody Waniliowe 1L	18.5	Mroźny Świat S.A.
16	Pizza Mrożona 4 Sery	12.5	Mroźny Świat S.A.
17	Jabłka Ligol	3.8	Sady Mazowieckie
18	Banany Premium	6.99	Sady Mazowieckie
19	Ryż Biały 1kg	4.99	Młyn i Spółka
20	Mąka Pszena 1kg	3.1	Młyn i Spółka
21	Kawa Ziarnista 500g	45	Herbaciarnia Świata
22	Herbata Czarna 100t	14.5	Herbaciarnia Świata
23	Pieprz Czarny	1.5	Przyprawy Orientu
24	Sól Morska 1kg	2.1	Przyprawy Orientu
25	Proszek do prania 3kg	34	Czysty Dom Hurt
26	Płyn do Naczyń	8.9	Czysty Dom Hurt
27	Żarówka LED E27	9.99	Techno-Sklep Hurtownia
28	Baterie AA (4szt)	15.99	Techno-Sklep Hurtownia
29	Krem do rąk	12.5	Piękno i Zdrowie
30	Szampon do włosów	15.2	Piękno i Zdrowie

2. Zestawienie produktów razem z ich alergenami (wyświetl też te bez alergenów)

```
SELECT
    p.nazwa AS produkt,
    pa.nazwa_alergenu AS alergen
FROM produkty p
LEFT OUTER JOIN produkty_alergeny pa ON p.id_produktu = pa.id_produktu
ORDER BY p.nazwa;
```

	PRODUKT	ALERGEN
1	Mleko 2% UHT	Laktoza
2	Szynka Konserwowa	Gorczyca
3	Szynka Konserwowa	Białko sojowe
4	Chleb Żytni	Gluten
5	Pomidory Malinowe	(null)
6	Jabłka Ligol	(null)
7	Lody Waniliowe 1L	Jaja
8	Lody Waniliowe 1L	Laktoza
9	Woda Gazowana 1.5L	(null)
10	Czekolada Mleczna	Laktoza
11	Czekolada Mleczna	Orzechy laskowe
12	Czekolada Mleczna	Lecytyna sojowa
13	Mąka Pszenna 1kg	Gluten
14	Kawa Ziarnista 500g	(null)
15	Płyn do Naczyń	(null)
16	Krem do rąk	(null)
17	Karma dla psa 5kg	Zboża
18	Pieprz Czarny	(null)
19	Baterie AA (4szt)	(null)
20	Jogurt Naturalny 400g	Laktoza
21	Bułka Kajzerka	Gluten
22	Kurczak Cały	Gorczyca
23	Marchew Polska	(null)
24	Banany Premium	(null)
25	Pizza Mrożona 4 Sery	Laktoza
26	Pizza Mrożona 4 Sery	Gluten
27	Sok Pomarańczowy 1L	(null)
28	Herbatniki Maślane	Gluten
29	Herbatniki Maślane	Jaja
30	Herbatniki Maślane	Laktoza
31	Ryż Biały 1kg	(null)
32	Herbata Czarna 100t	(null)
33	Proszek do prania 3kg	(null)
34	Szampon do włosów	(null)
35	Przysmak dla kota	Ryby
36	Sól Morska 1kg	(null)
37	Żarówka LED E27	(null)

3. Oblicza ile produktów jest na kg a ile na szt

```
SELECT
    PRODUKTY.JEDNOSTKA_MIARY,
    COUNT(*) AS LICZBA_PRODUKTÓW
FROM PRODUKTY
GROUP BY PRODUKTY.JEDNOSTKA_MIARY
```

	JEDNOSTKA_MIARY	LICZBA_PRODUKTÓW
1	szt	24
2	kg	6

4. Kasjerzy odpowiedzialni za kasy 2 i 4

```
SELECT
    p.IMIE,
    p.NAZWISKO
FROM PRACOWNICY p
JOIN KASJER k ON k.ID_PRACOWNIKA = p.ID_PRACOWNIKA
WHERE k.NR_KASY IN(2, 4)
```

	IMIE	NAZWISKO
1	Olga	Kaczmarek
2	Iga	Dudek
3	Nina	Wieczorek
4	Mariusz	Pudzianowski

5. Suma wydatków każdego klienta (Tylko ci, co wydali ponad 50 zł)

```
SELECT klienci_id_klienta, SUM(kwota) AS suma_zakupow
FROM zamowienia
GROUP BY klienci_id_klienta
HAVING SUM(kwota) > 50;
```

	KLIENCI_ID_KLIENTA	SUMA_ZAKUPOW
1	16	64.07
2	5	52.6
3	8	184.17

6. Średnia pensja na różnych stanowiskach (na podstawie przełożonego)

```
SELECT pracownicy_id_pracownika AS id_szeffa, ROUND(AVG(pensja), 2) AS
srednia_placa
FROM pracownicy
GROUP BY pracownicy_id_pracownika;
```

	ID_SZEFA	SREDNIA_PLACA
1	(null)	11200
2	1	7750
3	2	4380.54
4	3	4134.2

7. Szukanie produktów spożywczych zaczynających się od K lub M

```
SELECT nazwa, cena
FROM produkty
WHERE nazwa LIKE 'K%' or nazwa LIKE 'M%';
```

	NAZWA	CENA
1	Mleko 2% UHT	3.49
2	Mąka Pszenna 1kg	3.1
3	Kawa Ziarnista 500g	45
4	Krem do rąk	12.5
5	Karma dla psa 5kg	35
6	Kurczak Cały	18.9
7	Marchew Polska	3.5

8. Wybór zamówień opłaconych Blikiem lub Kartą

```
SELECT id_zamowienia, kwota, metoda_platnosci
FROM zamowienia
WHERE metoda_platnosci IN ('Blik', 'Karta');
```

	ID_ZAMOWIENIA	KWOTA	METODA_PLATNOSCI
1	1	11.18	Karta
2	2	15.99	Blik
3	4	43.1	Blik
4	5	18.9	Karta
5	7	37	Karta
6	8	18.19	Blik
7	9	180.37	Karta
8	11	27.3	Blik
9	12	45	Karta
10	14	21.25	Blik
11	15	20.97	Karta
12	17	16.7	Karta
13	19	22.1	Blik
14	20	7.6	Blik
15	21	19.56	Karta

9. Porównanie cen z kategorią "Nabiał"

```

SELECT
    nazwa,
    cena AS cena_produktu
FROM produkty
WHERE cena > ANY (SELECT cena FROM produkty WHERE kategorie_id_kategorii = 1)
ORDER BY cena ASC;

```

	NAZWA	CENA_PRODUKTU
1	Mąka Pszenna 1kg	3.1
2	Herbatniki Maślane	3.2
3	Mleko 2% UHT	3.49
4	Marchew Polska	3.5
5	Jabłka Ligoł	3.8
6	Chleb Żytni	4.2
7	Przysmak dla kota	4.5
8	Czekolada Mleczna	4.8
9	Ryż Biały 1kg	4.99
10	Sok Pomarańczowy 1L	5.8
11	Banany Premium	6.99
12	Płyn do Naczyń	8.9
13	Żarówka LED E27	9.99
14	Pizza Mrożona 4 Sery	12.5
15	Krem do rąk	12.5
16	Herbata Czarna 100t	14.5
17	Pomidory Malinowe	14.99
18	Szampon do włosów	15.2
19	Baterie AA (4szt)	15.99
20	Lody Waniliowe 1L	18.5
21	Kurczak Cały	18.9
22	Proszek do prania 3kg	34
23	Karma dla psa 5kg	35
24	Szynka Konserwowa	42.5
25	Kawa Ziarnista 500g	45

10. Pracownicy, którzy zarabiają więcej niż średnia w firmie

```
SELECT imie, nazwisko, pensja
FROM pracownicy
WHERE pensja > (SELECT AVG(pensja) FROM pracownicy);
```

	IMIE	NAZWISKO	PENSJA
1	Grzegorz	Rowak	11200
2	Paweł	Kowalski	7500
3	Krystyna	Ramecka	8000

11. Produkty, które należą do kategorii "Nabiał"

```
SELECT nazwa, cena
FROM produkty
WHERE kategorie_id_kategorii = (SELECT id_kategorii FROM kategorie WHERE nazwa = 'Nabiał i jaja');
```

	NAZWA	CENA
1	Mleko 2% UHT	3.49
2	Jogurt Naturalny 400g	2.8

12. Sprawdzenie, czy dany produkt był kiedykolwiek reklamowany

```
SELECT p.nazwa
FROM produkty p
WHERE EXISTS (
    SELECT 1
    FROM reklamacje r
    JOIN pozycje_zamowienia pz ON r.zamowienia_id_zamowienia =
pz.zamowienia_id_zamowienia
    WHERE pz.produkty_id_produkty = p.id_produkty
);
```

	NAZWA
1	Mleko 2% UHT
2	Szynka Konserwowa
3	Chleb Żytni
4	Pomidory Malinowe
5	Jabłka Ligol
6	Czekolada Mleczna
7	Mąka Pszenna 1kg
8	Kawa Ziarnista 500g
9	Płyn do Naczyń
10	Krem do rąk
11	Pieprz Czarny
12	Baterie AA (4szt)
13	Jogurt Naturalny 400g
14	Marchew Polska
15	Banany Premium
16	Pizza Mrożona 4 Sery
17	Sok Pomarańczowy 1L
18	Herbatniki Maślane
19	Herbata Czarna 100t
20	Proszek do prania 3kg
21	Sól Morska 1kg

13. Najdroższe zamówienie ze wszystkich

```
SELECT k.IMIE, k.NAZWISKO, z.ID_ZAMOWIENIA, z.KWOTA
FROM ZAMOWIENIA z
JOIN KLIENCI k ON z.KLIENCI_ID_KLIENTA = k.ID_KLIENTA
WHERE KWOTA >= ALL (SELECT KWOTA FROM ZAMOWIENIA);
```

	IMIE	NAZWISKO	ID_ZAMOWIENIA	KWOTA
1	Małgorzata	Woźniak	9	180.37

14. Produkty, których cena jest wyższa niż średnia cena w ich własnej kategorii

```
SELECT p1.NAZWA, p1.CENA
FROM PRODUKTY p1
WHERE p1.CENA > (
    SELECT AVG(p2.CENA)
    FROM PRODUKTY p2
    WHERE p2.KATEGORIE_ID_KATEGORII = p1.KATEGORIE_ID_KATEGORII
)
ORDER BY p1.CENA;
```

	NAZWA	CENA
1	Sól Morska 1kg	2.1
2	Mleko 2% UHT	3.49
3	Chleb Żytni	4.2
4	Czekolada Mleczna	4.8
5	Ryż Biały 1kg	4.99
6	Sok Pomarańczowy 1L	5.8
7	Banany Premium	6.99
8	Pomidory Malinowe	14.99
9	Szampon do włosów	15.2
10	Baterie AA (4szt)	15.99
11	Lody Waniliowe 1L	18.5
12	Proszek do prania 3kg	34
13	Karma dla psa 5kg	35
14	Szynka Konserwowa	42.5
15	Kawa Ziarnista 500g	45

15. Wyświetlenie listy alergenów dla produktów, które je posiadają:

```
SELECT p.nazwa, a.nazwa_alergenu
FROM produkty p
JOIN produkty_alergeny a ON p.id_produktu = a.id_produktu
WHERE a.nazwa_alergenu IS NOT NULL;
```

	NAZWA	NAZWA_ALERGENU
1	Mleko 2% UHT	Laktoza
2	Szynka Konserwowa	Gorczyca
3	Szynka Konserwowa	Białko sojowe
4	Chleb Żytni	Gluten
5	Lody Waniliowe 1L	Jaja
6	Lody Waniliowe 1L	Laktoza
7	Czekolada Mleczna	Laktoza
8	Czekolada Mleczna	Orzechy laskowe
9	Czekolada Mleczna	Lecytyna sojowa
10	Mąka Pszenna 1kg	Gluten
11	Karma dla psa 5kg	Zboża
12	Jogurt Naturalny 400g	Laktoza
13	Bułka Kajzerka	Gluten
14	Kurczak Cały	Gorczyca
15	Pizza Mrożona 4 Sery	Laktoza
16	Pizza Mrożona 4 Sery	Gluten
17	Herbatniki Maślane	Gluten
18	Herbatniki Maślane	Jaja
19	Herbatniki Maślane	Laktoza
20	Przysmak dla kota	Ryby

Procedury składowane

zlicz_produkty_wedlug_ceny

Wyszukuje produkty z zadanego przedziału cenowego, wyświetla je i zlicza.

```
CREATE OR REPLACE PROCEDURE zlicz_produkty_wedlug_ceny (p_min NUMBER, p_max
NUMBER) AS
    v_counter NUMBER := 0;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Szukam produktów w cenie od ' || p_min || ' do ' ||
p_max || ':');

```

```

FOR produkt IN (SELECT nazwa, cena FROM produkty) LOOP
    IF produkt.cena >= p_min AND produkt.cena <= p_max THEN
        v_counter := v_counter + 1;
        DBMS_OUTPUT.PUT_LINE('Znaleziono: ' || produkt.nazwa || ' (Cena: '
|| produkt.cena || ')');
    END IF;
END LOOP;
DBMS_OUTPUT.PUT_LINE('-----');
DBMS_OUTPUT.PUT_LINE('Łączna liczba produktów: ' || v_counter);
END;
/

```

```

SET SERVEROUTPUT ON;
EXEC zlicz_produkty_wedlug_ceny(1, 5);

```

```

Szukam produktów w cenie od 1 do 5:
Znaleziono: Mleko 2% UHT (Cena: 3,49)
Znaleziono: Chleb Żytni (Cena: 4,2)
Znaleziono: Jabłka Ligol (Cena: 3,8)
Znaleziono: Woda Gazowana 1.5L (Cena: 2,2)
Znaleziono: Czekolada Mleczna (Cena: 4,8)
Znaleziono: Mąka Pszena 1kg (Cena: 3,1)
Znaleziono: Pieprz Czarny (Cena: 1,5)
Znaleziono: Jogurt Naturalny 400g (Cena: 2,8)
Znaleziono: Marchew Polska (Cena: 3,5)
Znaleziono: Herbatniki Maślane (Cena: 3,2)
Znaleziono: Ryż Biały 1kg (Cena: 4,99)
Znaleziono: Przysmak dla kota (Cena: 4,5)
Znaleziono: Sól Morska 1kg (Cena: 2,1)
-----
Łączna liczba produktów: 13

```

przysnaj_premie

Przysnaj_premie pracownikowi, obsługuje wyjątki.

```

CREATE OR REPLACE PROCEDURE przysnaj_premie (p_id_prac NUMBER, p_premia
NUMBER) AS
    v_pensja NUMBER;
    e_premia_za_wysoka EXCEPTION;
BEGIN
    SELECT pensja INTO v_pensja FROM pracownicy WHERE id_pracownika =
p_id_prac;

    IF p_premia > v_pensja THEN
        RAISE e_premia_za_wysoka;
    END IF;

```

```

        DBMS_OUTPUT.PUT_LINE('Pracownik ID ' || p_id_prac || ' otrzymał premię: '
|| p_premia);

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Błąd: Nie ma pracownika o ID ' || p_id_prac);

    WHEN e_premia_zawysoka THEN
        DBMS_OUTPUT.PUT_LINE('Błąd: Premia (' || p_premia || ') nie może być
wyższa niż pensja!');

    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Wystąpił nieoczekiwany błąd.');
```

END;

/

```

SET SERVEROUTPUT ON;
EXEC przyznaj_premie(13, 500);
```

Pracownik ID 13 otrzymał premię: 500

PL/SQL procedure successfully completed.

```

SET SERVEROUTPUT ON;
EXEC przyznaj_premie(13, 5000);
```

Błąd: Premia (5000) nie może być wyższa niż pensja!

PL/SQL procedure successfully completed.

```

SET SERVEROUTPUT ON;
EXEC przyznaj_premie(9999, 500);
```

Błąd: Nie ma pracownika o ID 9999

PL/SQL procedure successfully completed.

Wyzwalacze

trg_walidacja_ceny

Sprawdza czy nowa cena nie jest niższa lub równa 0.

```
CREATE OR REPLACE TRIGGER trg_walidacja_ceny
BEFORE INSERT OR UPDATE ON produkty
FOR EACH ROW
BEGIN
    IF :NEW.cena <= 0 THEN
        RAISE_APPLICATION_ERROR(-20005, 'BŁĄD: Cena produktu musi być wyższa
niż 0 zł!');
    END IF;
END;
/
```

```
UPDATE produkty SET cena = -5 WHERE id_produktu = 1;
```

```
Error starting at line : 1 in command -
UPDATE produkty SET cena = -5 WHERE id_produktu = 1
Error at Command Line : 1 Column : 8
Error report -
SQL Error: ORA-20005: BŁĄD: Cena produktu musi być wyższa niż 0 zł!
ORA-06512: at "SKLEP_PROJEKT.TRG_WALIDACJA_CENY", line 3
ORA-04088: error during execution of trigger 'SKLEP_PROJEKT.TRG_WALIDACJA_CENY'

https://docs.oracle.com/error-help/db/ora-20005/

More Details :
https://docs.oracle.com/error-help/db/ora-20005/
https://docs.oracle.com/error-help/db/ora-06512/
https://docs.oracle.com/error-help/db/ora-04088/
```

trg_limit_podwyzki

Sprawdza czy zmiana pensji pracownika nie jest zbyt duża (ponad 1000 zł).

```
CREATE OR REPLACE TRIGGER trg_limit_podwyzki
BEFORE UPDATE OF pensja ON pracownicy
FOR EACH ROW
BEGIN
    IF :NEW.pensja - :OLD.pensja > 1000 THEN
        RAISE_APPLICATION_ERROR(-20006, 'BŁĄD: Jednorazowa podwyżka nie może
przekraczać 1000 zł!');
```

```

    END IF;
END;
/

```

```
UPDATE pracownicy SET pensja = pensja + 2000 WHERE id_pracownika = 10;
```

```

Error starting at line : 1 in command -
UPDATE pracownicy SET pensja = pensja + 2000 WHERE id_pracownika = 10
Error at Command Line : 1 Column : 8
Error report -
SQL Error: ORA-20006: BŁĄD: Jednorazowa podwyżka nie może przekraczać 1000 zł!
ORA-06512: at "SKLEP_PROJEKT.TRG_LIMIT_PODWYZKI", line 3
ORA-04088: error during execution of trigger 'SKLEP_PROJEKT.TRG_LIMIT_PODWYZKI'

https://docs.oracle.com/error-help/db/ora-20006/

More Details :
https://docs.oracle.com/error-help/db/ora-20006/
https://docs.oracle.com/error-help/db/ora-06512/
https://docs.oracle.com/error-help/db/ora-04088/

```

trg_oznacz_promocja

Zaznacza produkty w promocyjnej cenie.

```

CREATE OR REPLACE TRIGGER trg_oznacz_promocje
BEFORE UPDATE OF cena ON produkty
FOR EACH ROW
BEGIN
    IF :NEW.cena < :OLD.cena THEN
        :NEW.nazwa := :NEW.nazwa || ' (PROMOCJA)';
    END IF;
END;
/

```

```
SELECT nazwa, cena FROM produkty WHERE id_produktu = 10;
```

	NAZWA	CENA
1	Kawa Ziarnista 500g	45

```
UPDATE produkty SET cena = cena * 0.8 WHERE id_produktu = 10;
```

Jeszcze raz:

```
SELECT nazwa, cena FROM produkty WHERE id_produktu = 10;
```

	NAZWA	CENA
1	Kawa Ziarnista 500g (PROMOCJA)	36

Aplikacja kliencka

main_gui.py

```
#!/usr/bin/env python3

import tkinter as tk
from tkinter import ttk, messagebox, Toplevel
import db_operations
from report_manager import ReportManager

class SklepApp:
    def __init__(self, root):
        """Inicjalizacja głównego okna aplikacji oraz konfiguracja systemu
        zakładek nawigacyjnych."""
        self.root = root
        self.root.title("System Zarządzania Sklepem")
        self.root.geometry("1000x600")

        # Implementacja kontenera zakładek do separacji widoków
        # poszczególnych tabel
        self.notebook = ttk.Notebook(self.root)
        self.notebook.pack(expand=True, fill='both')

        # Inicjalizacja menedżera raportów
        self.report_mgr = ReportManager()

        # Wykaz encji bazy danych podlegających obsłudze w interfejsie
        # użytkownika
        tabele = ["KATEGORIE", "DOSTAWCY", "KLIENCI", "MAGAZYNY",
        "PRODUKTY"]

        for nazwa in tabele:
            frame = ttk.Frame(self.notebook)
            self.notebook.add(frame, text=nazwa)
            self.setup_table_tab(frame, nazwa)

        self.tab_reports = ttk.Frame(self.notebook)
        self.notebook.add(self.tab_reports, text="RAPORTY")
        self.setup_reports_tab()

    def setup_table_tab(self, frame, table_name):
        """Konfiguracja panelu wyszukiwania, przycisków sterujących oraz
        komponentu prezentacji danych."""
        # --- Panel Wyszukiwania i Kryteriów ---
        search_frame = ttk.LabelFrame(frame, text="Wyszukiwanie
```

```

(Kryteria)")
    search_frame.pack(side="top", fill="x", padx=10, pady=5)

    ttk.Label(search_frame, text="Kolumna:").pack(side="left", padx=5,
pady=5)
    # Komponent wyboru kolumny do filtrowania danych
    combo_search_col = ttk.Combobox(search_frame, state="readonly",
width=20)
    combo_search_col.pack(side="left", padx=5, pady=5)

    ttk.Label(search_frame, text="Fraza:").pack(side="left", padx=5,
pady=5)
    ent_search_val = ttk.Entry(search_frame, width=30)
    ent_search_val.pack(side="left", padx=5, pady=5)

    # --- Panel Przycisków Operacyjnych ---
    btn_frame = ttk.Frame(frame)
    btn_frame.pack(side="top", fill="x", padx=10, pady=5)

    # Inicjalizacja komponentu Treeview do wyświetlania rekordów z bazy
    tree = ttk.Treeview(frame, columns=[], show='headings')
    tree.pack(expand=True, fill='both', padx=10, pady=5)

    # Definicja procedury realizującej wyszukiwanie dynamiczne
    def do_search():
        col = combo_search_col.get()
        val = ent_search_val.get()
        self.load_data(tree, table_name, col, val)

    # Definicja procedury przywracającej pełny widok danych
    def do_reset():
        ent_search_val.delete(0, tk.END)
        self.load_data(tree, table_name)

    ttk.Button(search_frame, text="Szukaj",
command=do_search).pack(side="left", padx=5)
    ttk.Button(search_frame, text="Resetuj",
command=do_reset).pack(side="left", padx=5)

    # Implementacja przycisków wywołujących operacje CRUD i odświeżanie
    ttk.Button(btn_frame, text="Odśwież",
        command=lambda: self.load_data(tree,
table_name)).pack(side="left")

    # Warunkowy wybór okna formularza w zależności od typu tabeli
    if table_name == "PRODUKTY":
        ttk.Button(btn_frame, text="Dodaj nowy",
            command=lambda:
self.open_add_product_window(tree)).pack(side="left", padx=5)
    else:
        ttk.Button(btn_frame, text="Dodaj nowy",
            command=lambda:
self.open_add_generic_window(table_name, tree)).pack(side="left", padx=5)

    ttk.Button(btn_frame, text="Usuń zaznaczone",
        command=lambda: self.delete_selected(tree,
table_name)).pack(side="left")

    # Inicjalne pobranie danych oraz mapowanie nazw kolumn do systemu
wyszukiwania
    self.load_data(tree, table_name)

```

```

        combo_search_col['values'] = tree["columns"]
        if tree["columns"]:
            combo_search_col.current(1) # Domyślny wybór drugiej kolumny
            (zazwyczaj atrybut opisowy)

    def load_data(self, tree, table_name, search_col=None,
search_val=None):
        """Komunikacja z modulem db_operations w celu aktualizacji
zawartości widoku danych."""
        cols, data = db_operations.fetch_all(table_name, search_col,
search_val)

        # Konfiguracja struktury kolumn na podstawie deskryptorów z bazy
danych
        tree["columns"] = cols
        for col in cols:
            tree.heading(col, text=col)
            tree.column(col, width=100)

        # Usunięcie nieaktualnych wpisów z interfejsu
        for item in tree.get_children():
            tree.delete(item)

        # Proces zasilania komponentu graficznego nowymi rekordami
        for row in data:
            tree.insert("", "end", values=row)

    def open_add_product_window(self, tree):
        """Inicjalizacja dedykowanego formularza dla tabeli PRODUKTY z
obsługą mapowania kluczy obcych."""
        top = Toplevel(self.root)
        top.title("Dodaj nowy produkt")

        # Kontener główny zapewniający separację elementów od krawędzi okna
        main_frame = ttk.Frame(top, padding="20 20 20 20")
        main_frame.pack(expand=True, fill="both")

        SZEROKOSC_POLA = 50

        # Pobranie danych referencyjnych dla list rozwijanych (mapowanie
Nazwa -> ID)
        dostawcy_map = db_operations.get_lookup_data("DOSTAWCY",
"ID_DOSTAWCY",
"NAZWA")
        magazyny_map = db_operations.get_magazyny_lookup()
        kategorie_map = db_operations.get_lookup_data("KATEGORIE",
"ID_KATEGORII",
"NAZWA")

        # Definicja etykiet i pól tekstowych dla atrybutów podstawowych
        ttk.Label(main_frame, text="NAZWA PRODUKTU:").pack(pady=2,
anchor="w")
        ent_nazwa = ttk.Entry(main_frame, width=SZEROKOSC_POLA)
        ent_nazwa.pack()

        ttk.Label(main_frame, text="CENA:").pack(pady=2, anchor="w")
        ent_cena = ttk.Entry(main_frame, width=SZEROKOSC_POLA)
        ent_cena.pack()

        ttk.Label(main_frame, text="JEDNOSTKA (szt/kg):").pack(pady=2,
anchor="w")

```

```

ent_jednostka = ttk.Entry(main_frame, width=SZEROKOSC_POLA)
ent_jednostka.pack()

# Konfiguracja komponentów Combobox zapewniających poprawność
więzów integralności (klucze obce)
ttk.Label(main_frame, text="DOSTAWCA:").pack(pady=2, anchor="w")
combo_dostawca = ttk.Combobox(main_frame,
                               values=list(dostawcy_map.keys()),
                               state="readonly",
                               width=SZEROKOSC_POLA - 3)

combo_dostawca.pack()

ttk.Label(main_frame, text="KATEGORIA:").pack(pady=2, anchor="w")
combo_kategoria = ttk.Combobox(main_frame,
                                values=list(kategorie_map.keys()),
                                state="readonly",
                                width=SZEROKOSC_POLA - 3)

combo_kategoria.pack()

ttk.Label(main_frame, text="MAGAZYN:").pack(pady=2, anchor="w")
combo_magazyn = ttk.Combobox(main_frame,
                              values=list(magazyny_map.keys()),
                              state="readonly",
                              width=SZEROKOSC_POLA - 3)

combo_magazyn.pack()

# Obsługa atrybutu wielowartościowego wprowadzanego jako ciąg
znaków
ttk.Label(main_frame, text="ALERGENY (rozdzielone
przecinkiem):").pack(pady=2, anchor="w")
ent_alergeny = ttk.Entry(main_frame, width=SZEROKOSC_POLA)
ent_alergeny.pack()

def save():
    """Proces gromadzenia danych, walidacji typów oraz wywołania
    procedury wstawiania do bazy."""
    try:
        # Transformacja danych z formularza na strukturę słownikową
        z uwzględnieniem mapowania ID
        data = {
            'NAZWA': ent_nazwa.get(),
            'CENA': float(ent_cena.get()),
            'JEDNOSTKA_MIARY': ent_jednostka.get(),
            'KOD_KRESKOWY': "",
            'DOSTAWCA_ID': dostawcy_map[combo_dostawca.get()],
            'MAGAZYN_ID': magazyny_map[combo_magazyn.get()],
            'KATEGORIA_ID': kategorie_map[combo_kategoria.get()]
        }

        if db_operations.insert_product(data, ent_alergeny.get()):
            messagebox.showinfo("Sukces",
                                "Operacja zakończona powodzeniem.")
            top.destroy()
            self.load_data(tree, "PRODUKTY")
        else:
            messagebox.showerror("Błąd",
                                "Błąd podczas zapisu w bazie
danych.")
    except Exception as e:
        messagebox.showerror("Błąd walidacji",
                                f"Niepoprawne dane: {e}")

```

```

        ttk.Button(main_frame, text="Zapisz", command=save).pack(pady=20)

    def open_add_generic_window(self, table_name, tree):
        """Inicjalizacja dynamicznego okna formularza dla tabel o prostych
        strukturach atrybutów."""
        top = Toplevel(self.root)
        top.title(f"Dodaj: {table_name}")

        # Konfiguracja kontenera z marginesami wewnętrznymi
        main_frame = ttk.Frame(top, padding="20 20 20 20")
        main_frame.pack(expand=True, fill="both")

        # Mapowanie wymaganych pól wejściowych dla poszczególnych tabel
        fields_config = {
            "KATEGORIE": ["NAZWA"],
            "DOSTAWCY": ["NAZWA", "NIP", "NR_TEL", "ADRES"],
            "KLIENCI": ["IMIE", "NAZWISKO", "NR_TEL", "EMAIL",
                       "KARTA_RABATOWA"],
            "MAGAZYNY": ["SEGMENT", "OPIS"]
        }

        fields = fields_config.get(table_name, [])
        entries = {}

        # Automatyczne generowanie etykiet i pól tekstowych na podstawie
        # konfiguracji
        for field in fields:
            ttk.Label(main_frame, text=f"{field}:").pack(pady=2,
            anchor="w")
            ent = ttk.Entry(main_frame, width=40)
            ent.pack(pady=5, fill="x")
            entries[field] = ent

        def save():
            """Gromadzenie danych z wygenerowanych pól i wywołanie
            uniwersalnej procedury zapisu."""
            data = {field: ent.get() for field, ent in entries.items()}
            if db_operations.insert_generic(table_name, data):
                messagebox.showinfo("Sukces", "Rekord dodany.")
                top.destroy()
                self.load_data(tree, table_name)
            else:
                messagebox.showerror("Błąd", "Nie udało się zapisać
                danych.")

        ttk.Button(main_frame, text="Zapisz", command=save).pack(pady=15)

    def delete_selected(self, tree, table_name):
        """Procedura weryfikacji zaznaczenia oraz wywołania operacji
        usuwania rekordu z bazy danych."""
        selected_item = tree.selection()

        # Walidacja wystąpienia zdarzenia zaznaczenia elementu w widoku
        if not selected_item:
            messagebox.showwarning("Brak wyboru",
            "Proszę najpierw zaznaczyć rekord do
            usunięcia.")
            return

        # Ekstrakcja danych z zaznaczonego wiersza w celu identyfikacji

```

```

klucza głównego
    item_values = tree.item(selected_item)['values']
    id_val = item_values[0]
    id_col = tree["columns"][0]

    # Wymagane potwierdzenie przed nieodwracalną zmianą stanu bazy
danych
    if messagebox.askyesno("Potwierdzenie",
                           f"Czy na pewno usunąć rekord o ID:
{id_val}?"):
        try:
            # Wywołanie procedury usuwania z obsługą integralności w
db_operations
            db_operations.delete_record(table_name, id_col, id_val)
            messagebox.showinfo("Sukces", "Rekord został usunięty.")
            # Aktualizacja interfejsu w celu odzwierciedlenia zmian w
bazie
            self.load_data(tree, table_name)
        except Exception as e:
            messagebox.showerror("Błąd",
                                f"Nie udało się usunąć rekordu: {e}")

def setup_reports_tab(self):
    frame = ttk.Frame(self.tab_reports, padding="20")
    frame.pack(expand=True, fill="both")

    # Raport Produktów (Grupowanie + 2 kryteria)
    sect1 = ttk.LabelFrame(frame, text="Raport Produktów",
                           padding="10")
    sect1.pack(fill="x", pady=5)
    ttk.Label(sect1, text="Cena min:").grid(row=0, column=0)
    self.ent_min = ttk.Entry(sect1, width=10);
    self.ent_min.grid(row=0, column=1);
    self.ent_min.insert(0, "0")
    ttk.Label(sect1, text="Cena max:").grid(row=0, column=2)
    self.ent_max = ttk.Entry(sect1, width=10);
    self.ent_max.grid(row=0, column=3);
    self.ent_max.insert(0, "10")
    ttk.Button(sect1, text="Generuj",
               command=lambda: self.report_mgr.raport_produkty_ceny(
                   self.ent_min.get(), self.ent_max.get())).grid(row=0,
column=4,
padx=10)

    # Statystyka (Wykres)
    sect2 = ttk.LabelFrame(frame, text="Statystyka Magazynu",
                           padding="10")
    sect2.pack(fill="x", pady=5)
    ttk.Button(sect2, text="Generuj Raport z Wykresem",
command=self.report_mgr.raport_statystyka_magazynu).pack()

    # Formularz Klienta
    sect3 = ttk.LabelFrame(frame, text="Karta Klienta",
                           padding="10")
    sect3.pack(fill="x", pady=5)
    ttk.Label(sect3, text="Nazwisko klienta:").grid(row=0, column=0)
    self.ent_klient = ttk.Entry(sect3);
    self.ent_klient.grid(row=0, column=1)

```



```

        ttk.Button(sect3, text="Generuj Kartę",
                    command=lambda: self.report_mgr.raport_karta_klienta(
                        self.ent_klient.get())).grid(row=0, column=2,
padx=10)

        # Lista Dostawców
        sect4 = ttk.LabelFrame(frame, text="Lista Dostawców",
                               padding="10")
        sect4.pack(fill="x", pady=5)
        ttk.Button(sect4, text="Generuj Listę",
                    command=self.report_mgr.raport_lista_dostawcow).pack()

if __name__ == "__main__":
    # Start pętli głównej aplikacji
    root = tk.Tk()
    app = SklepApp(root)
    root.mainloop()

```

db_operations.py

```

#!/usr/bin/env python3

import oracledb

# Parametry konfiguracyjne niezbędne do nawiązania sesji z serwerem bazy
danych Oracle
DB_CONFIG = {
    "user": "sklep_projekt",
    "password": "Klarnetjac21",
    "dsn": "localhost:1521/XEPDB1"
}

def get_connection():
    """Inicjalizuje i zwraca obiekt połączenia z bazą danych przy użyciu
sterownika oracledb."""
    return oracledb.connect(**DB_CONFIG)

def fetch_all(table_name, search_col=None, search_val=None):
    """
    Realizuje pobieranie rekordów z bazy danych. W przypadku tabeli
PRODUKTY
    implementuje zaawansowane złączenia (JOIN) oraz agregację danych
wielowartościowych (LISTAGG).
    Obsługuje dynamiczne filtrowanie wyników na podstawie przekazanych
parametrów.
    """
    conn = get_connection()
    cursor = conn.cursor()
    params = []

    # Implementacja logiki zapytania dla tabeli PRODUKTY z uwzględnieniem
kluczy obcych i alergenów
    if table_name.upper() == "PRODUKTY":
        sql = """
            SELECT p.ID_PRODUKTU, p.NAZWA, p.CENA, p.JEDNOSTKA_MIARY,
p.KOD_KRESKOWY,
            d.NAZWA as DOSTAWCA, m.SEGMENT as MAGAZYN, k.NAZWA as

```

```

KATEGORIA,
        (SELECT LISTAGG (NAZWA_ALERGENU, ', ') WITHIN GROUP (ORDER
BY NAZWA_ALERGENU)
        FROM PRODUKTY_ALERGENY pa WHERE pa.ID_PRODUKTU =
p.ID_PRODUKTU) as ALERGENY
        FROM PRODUKTY p
        JOIN DOSTAWCY d ON p.DOSTAWCY_ID_DOSTAWCY = d.ID_DOSTAWCY
        JOIN MAGAZYNY m ON p.MAGAZYNY_ID_MAGAZYNU = m.ID_MAGAZYNU
        JOIN KATEGORIE k ON p.KATEGORIE_ID_KATEGORII = k.ID_KATEGORII
    """
    if search_col and search_val:
        # Rozszerzenie zapytania o klauzulę WHERE dla celów filtrowania
frazowego
        sql += f" WHERE p.{search_col} LIKE :1"
        params.append(f"%{search_val}%")
    else:
        # Standardowe zapytanie typu SELECT dla pozostałych encji bazy
danych
        sql = f"SELECT * FROM {table_name}"
        if search_col and search_val:
            sql += f" WHERE {search_col} LIKE :1"
            params.append(f"%{search_val}%")

    cursor.execute(sql, params)
    data = cursor.fetchall()
    # Wydobycie nazw kolumn
    cols = [col[0] for col in cursor.description]
    cursor.close()
    conn.close()
    return cols, data

def get_lookup_data(table, id_col, name_col):
    """Pobiera pary danych identyfikator-nazwa wykorzystywane do
Combobox."""
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(f"SELECT {id_col}, {name_col} FROM {table}")
    # Mapowanie wyników do słownika
    lookup = {row[1]: row[0] for row in cursor}
    cursor.close()
    conn.close()
    return lookup

def get_magazyny_lookup():
    """
    Pobiera dane z tabeli MAGAZYNY i formatuje je w sposób czytelny dla
operatora,
    łącząc nazwę segmentu z opcjonalnym opisem lokalizacji.
    """
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute("SELECT ID_MAGAZYNU, SEGMENT, OPIS FROM MAGAZYNY")

    lookup = {}
    for row in cursor:
        id_mag, segment, opis = row
        # Warunkowe formatowanie ciągu znaków wyświetlanego w interfejsie
użytkownika
        tekst_wyswietlany = f"{segment} ({opis})" if opis else segment

```

```

        lookup[tekst_wyswietlany] = id_mag

    cursor.close()
    conn.close()
    return lookup

def insert_product(data, alergens_str):
    """
    Realizuje operację wstawiania produktu oraz powiązanych z nim
    alergenów.
    Zapewnia integralność operacji poprzez wykorzystanie mechanizmu
    transakcji (commit/rollback).
    """
    conn = get_connection()
    cursor = conn.cursor()
    try:
        # Wstawienie danych do tabeli PRODUKTY z wykorzystaniem klauzuli
        RETURNING dla klucza głównego
        sql_prod = """
            INSERT INTO PRODUKTY (NAZWA, CENA, JEDNOSTKA_MIARY,
                                KOD_KRESKOWY,
                                DOSTAWCY_ID_DOSTAWCY,
                                MAGAZYNY_ID_MAGAZYNU,
                                KATEGORIE_ID_KATEGORII)
            VALUES (:1, :2, :3, :4, :5, :6, :7)
            RETURNING ID_PRODUKTU INTO :8
        """

        new_id_var = cursor.var(int)

        # Obsługa opcjonalności atrybutu KOD_KRESKOWY
        kod = data['KOD_KRESKOWY'] if data['KOD_KRESKOWY'].strip() else
None

        cursor.execute(sql_prod, [
            data['NAZWA'], data['CENA'], data['JEDNOSTKA_MIARY'], kod,
            data['DOSTAWCA_ID'], data['MAGAZYN_ID'], data['KATEGORIA_ID'],
            new_id_var
        ])
        new_id = new_id_var.getvalue()[0]

        # Iteracyjne wstawianie rekordów do tabeli asocjacyjnej
        PRODUKTY_ALERGENY
        if alergens_str.strip():
            alergen_y = [a.strip() for a in alergens_str.split(",")]
            for alergen in alergen_y:
                cursor.execute(
                    "INSERT INTO PRODUKTY_ALERGENY (NAZWA_ALERGENU,
ID_PRODUKTU) VALUES (:1, :2)",
                    [alergen, new_id]
                )

            conn.commit()
            return True
        except Exception as e:
            # Wycofanie wszystkich zmian w przypadku wystąpienia błędu podczas
            zapisu
            conn.rollback()
            print(f"Błąd operacji INSERT: {e}")
            return False
    finally:

```

```

        cursor.close()
        conn.close()

def insert_generic(table_name, data_dict):
    """
    Uniwersalny mechanizm wstawiania rekordów dla tabel o prostej
    strukturze danych.
    Dynamicznie buduje polecenie INSERT na podstawie struktury przekazanego
    słownika.
    """
    conn = get_connection()
    cursor = conn.cursor()

    # Konwersja pustych ciągów tekstowych na wartości NULL w celu
    zachowania spójności danych
    for key in data_dict:
        if isinstance(data_dict[key], str) and not data_dict[key].strip():
            data_dict[key] = None

    cols = ", ".join(data_dict.keys())
    placeholders = ", ".join([f":{i + 1}" for i in range(len(data_dict))])
    values = list(data_dict.values())

    sql = f"INSERT INTO {table_name} ({cols}) VALUES ({placeholders})"

    try:
        cursor.execute(sql, values)
        conn.commit()
        return True
    except Exception as e:
        conn.rollback()
        print(f"Błąd INSERT w tabeli {table_name}: {e}")
        return False
    finally:
        cursor.close()
        conn.close()

def delete_record(table_name, id_col, id_val):
    """
    Usuwa wskazany rekord z bazy danych. Implementuje ręczne usuwanie
    kaskadowe
    dla powiązanych rekordów w tabeli alergenów przed usunięciem produktu
    nadrzędnego.
    """
    conn = get_connection()
    cursor = conn.cursor()
    try:
        if table_name.upper() == "PRODUKTY":
            # Usunięcie zależności wynikających z więzów integralności
            (klucz obcy)
            cursor.execute(
                "DELETE FROM PRODUKTY_ALERGENY WHERE ID_PRODUKTU = :1",
                [id_val])

            cursor.execute(f"DELETE FROM {table_name} WHERE {id_col} = :1",
                           [id_val])
            conn.commit()
    finally:

```

```

cursor.close()
conn.close()

```

report_manager.py

```

import os
from fpdf import FPDF
import matplotlib.pyplot as plt
import db_operations
from matplotlib.ticker import MaxNLocator

class ReportManager:
    """Klasa odpowiedzialna za generowanie wszystkich raportów."""

    def __init__(self, output_dir="output"):
        """Inicjalizacja ścieżek czcionek oraz folderu wyjściowego dla
        gotowych dokumentów."""
        self.output_dir = output_dir
        # Wykorzystanie czcionek systemowych TrueType (TTF) w celu
        # poprawnego renderowania polskich znaków
        self.font_path = r"C:\Windows\Fonts\arial.ttf"
        self.font_bold_path = r"C:\Windows\Fonts\arialbd.ttf"

        # Automatyczne tworzenie folderu wyjściowego, jeśli nie istnieje w
        # katalogu projektu
        if not os.path.exists(self.output_dir):
            os.makedirs(self.output_dir)

    def _setup_pdf(self):
        """Pomocnicza metoda konfigurująca obiekt FPDF, rejestrująca
        czcionkę Unicode wspierającą polskie."""
        pdf = FPDF()
        # Dodanie zewnętrznych czcionek TTF do rejestru biblioteki fpdf2
        # (PolishArial)
        pdf.add_font("PolishArial", style="", fname=self.font_path)
        pdf.add_font("PolishArial", style="B", fname=self.font_bold_path)
        pdf.add_page()
        return pdf

    def _save_and_open(self, pdf, filename):
        """Finalizacja zapisu pliku PDF i automatyczne wywołanie systemowej
        przeglądarki dokumentów."""
        path = os.path.join(self.output_dir, filename)
        pdf.output(path)
        os.startfile(path)

    def raport_produkty_ceny(self, min_c, max_c):
        """Generuje raport produktów z podziałem na kategorie (grupowanie)
        w zadanym zakresie cenowym."""
        try:
            # Pobranie danych przy użyciu zdefiniowanych złączeń (JOIN) z
            # modułu db_operations
            cols, data = db_operations.fetch_all("PRODUKTY")

            # Filtrowanie danych na podstawie 2 kryteriów (Cena Min/Max)
            filtered = [r for r in data if
                        float(min_c) <= float(r[2]) <= float(max_c)]

            # Przygotowanie danych do grupowania poprzez sortowanie po

```

```

nazwie kategorii (indeks 7)
    filtered.sort(key=lambda x: x[7])

    pdf = self._setup_pdf()
    pdf.set_font("PolishArial", "B", 16)
    pdf.cell(0, 10, f"Produkty w zakresie {min_c} - {max_c} PLN",
             ln=True, align='C')

    current_cat = None
    for row in filtered:
        # Logika wykrywania zmiany kategorii i wstawiania nagłówka
        if row[7] != current_cat:
            pdf.ln(5)
            pdf.set_font("PolishArial", "B", 12)
            pdf.set_fill_color(240, 240, 240)
            pdf.cell(0, 10, f"KATEGORIA: {row[7]}", ln=True,
                    fill=True)
            current_cat = row[7]

            pdf.set_font("PolishArial", "", 10)
            pdf.cell(0, 8, f" - {row[1]} | Cena: {row[2]} PLN",
                    ln=True)

        self._save_and_open(pdf, "raport_produkty.pdf")
        return True, "Sukces"
    except Exception as e:
        return False, str(e)

def raport_statystyka_magazynu(self):
    """Tworzy raport analityczny zawierający wizualizację danych w
    formie wykresu słupkowego."""
    try:
        # Zliczanie wystąpień produktów w każdej kategorii
        cols, data = db_operations.fetch_all("PRODUKTY")
        counts = {}
        for r in data:
            kat = r[7]
            counts[kat] = counts.get(kat, 0) + 1

        # Konfiguracja wykresu w bibliotece Matplotlib
        plt.figure(figsize=(11, 8))
        plt.bar(counts.keys(), counts.values(), color='orange')
        plt.title("Liczba produktów w kategoriach", fontsize=14,
                  fontweight='bold')
        plt.ylabel("Ilość sztuk")

        # Formatowanie osi Y: wymuszenie wyświetlania tylko liczb
        # całkowitych
        plt.gca().yaxis.set_major_locator(MaxNLocator(integer=True))

        # Formatowanie osi X: pionowy obrót etykiet w celu uniknięcia
        # nakładania się tekstu
        plt.xticks(rotation=90)
        plt.tight_layout()

        # Zapis grafiki do pliku tymczasowego przed osadzeniem w PDF
        chart_path = os.path.join(self.output_dir,
                                   "raport_produkty_wykres.png")
        plt.savefig(chart_path)
        plt.close()

```

```

        pdf = self._setup_pdf()
        pdf.set_font("PolishArial", "B", 20)
        pdf.cell(0, 20, "Statystyka Kategorii", ln=True, align='C')

        # Osadzenie wygenerowanego wykresu w dokumencie PDF
        pdf.image(chart_path, x=10, y=40, w=190)

        self._save_and_open(pdf, "raport_wykres_magazynu.pdf")
        return True, "Sukces"
    except Exception as e:
        return False, str(e)

def raport_karta_klienta(self, nazwisko_klienta):
    """Generuje kartę informacyjną wybranego klienta."""
    try:
        cols, data = db_operations.fetch_all("KLIENCI")
        # Wyszukiwanie klienta na podstawie frazy nazwiska (kryterium
        wyszukiwania)
        klient = next((r for r in data if
                        nazwisko_klienta.lower() in str(r[2]).lower()),
                       None)

        pdf = self._setup_pdf()
        # Rysowanie ramki
        pdf.rect(10, 10, 190, 80)
        pdf.set_font("PolishArial", "B", 14)
        pdf.cell(0, 15, "KARTA INFORMACYJNA KLIENTA", ln=True,
        align='C')

        pdf.set_font("PolishArial", "", 12)
        if klient:
            pdf.ln(5)
            pdf.cell(0, 10, f"ID Klienta: {klient[0]}", ln=True)
            pdf.cell(0, 10, f"Imię i Nazwisko: {klient[1]}
            {klient[2]}",
                    ln=True)
            pdf.cell(0, 10, f"Email: {klient[4]}", ln=True)
            pdf.cell(0, 10, f"Telefon: {klient[3]}", ln=True)
        else:
            pdf.cell(0, 10, "Nie znaleziono klienta o podanym
            nazwisku.",
                    ln=True)

        self._save_and_open(pdf, "raport_formularz_klienta.pdf")
        return True, "Sukces"
    except Exception as e:
        return False, str(e)

def raport_lista_dostawcow(self):
    """Generuje przejrzyste zestawienie wszystkich dostawców zapisanych
    w bazie danych."""
    try:
        cols, data = db_operations.fetch_all("DOSTAWCY")
        pdf = self._setup_pdf()
        pdf.set_font("PolishArial", "B", 14)
        pdf.cell(0, 10, "LISTA DOSTAWCÓW", ln=True)
        pdf.ln(5)

        pdf.set_font("PolishArial", "", 10)
        for r in data:

```

```

        # Wyświetlanie danych
        pdf.cell(0, 8, f"Firma: {r[1]} (NIP: {r[2]}) - Tel:
{r[3]}",
                ln=True, border='B')

        self._save_and_open(pdf, "lista_dostawcow.pdf")
        return True, "Sukces"
    except Exception as e:
        return False, str(e)

```

Opis programu

Aplikacja to kompletny system do zarządzania sklepem spożywczym, łączący bazę danych Oracle XE z interfejsem w języku Python. Program składa się z trzech współpracujących modułów: `db_operations.py` zarządza komunikacją z bazą i operacjami SQL, `main_gui.py` udostępnia okienkowy interfejs użytkownika z systemem zakładek i formularzy, natomiast `report_manager.py` odpowiada za generowanie raportów PDF z wykresami i grupowaniem danych. System w pełni obsługuje polskie znaki oraz zapewnia integralność danych dzięki wykorzystaniu złączeń tabel i list rozwijanych w formularzach.

Integracja raportowania z aplikacją

Integracja raportowania z aplikacją polega na połączeniu interfejsu graficznego z dedykowaną klasą `ReportManager`, która pełni rolę pośrednika między użytkownikiem a bazą danych. Po wybraniu konkretnego raportu w menu aplikacji, system przesyła wybrane kryteria do menedżera, który pobiera aktualne dane z bazy Oracle poprzez gotowe zapytania SQL zawarte w module `db_operations`. Następnie dane te są przetwarzane i formatowane do dokumentu PDF przy użyciu biblioteki `fpdf2`, a wszelkie statystyki są wizualizowane za pomocą wykresów generowanych w module `matplotlib`. Dzięki zastosowaniu mechanizmu dynamicznego ładowania czcionek systemowych `TrueType`, raporty poprawnie obsługują polskie znaki, a gotowy plik otwiera się automatycznie w domyślnej przeglądarce zaraz po wygenerowaniu.

Interfejs (zakładkami)

KATEGORIE

System Zarządzania Sklepem

KATEGORIE DOSTAWCY KLIENCI MAGAZYNY PRODUKTY RAPORTY

Wyszukiwanie (Kryteria)

Kolumna: NAZWA Fraz: Szukaj Resetuj

Odśwież Dodaj nowy Usuń zaznaczone

	ID_KATEGORII	NAZWA
1		Nabiał i jaja
2		Pieczycwo i wyroby cukiernicze
3		Mięso, drób i wędliny
4		Warzywa świeże
5		Owoce świeże
6		Mrożonki i dania gotowe
7		Napoje, wody i soki
8		Słodczye i słone przekąski
9		Produkty sypkie (mąki, kasze, makarony)
10		Kawy, herbaty i kakao
11		Chemia domowa i środki czystości
12		Kosmetyki i higiena osobista
13		Karma i akcesoria dla zwierząt
14		Przyprawy i dodatki do dań
15		Artykuły przemysłowe i sezonowe

DOSTAWCY

System Zarządzania Sklepem

KATEGORIE DOSTAWCY KLIENCI MAGAZYNY PRODUKTY RAPORTY

Wyszukiwanie (Kryteria)

Kolumna: NAZWA Fraz: Szukaj Resetuj

Odśwież Dodaj nowy Usuń zaznaczone

	ID_DOSTAWCY	NAZWA	NIP	NR_TEL	ADRES
1		Zakłady Mięsne Dobropol	5554443322	321115566	ul. Masarska 8, Katowice
2		Słodka Dolina	2223334455	613334455	None
3		Polmlek Sp. z o.o.	1234567890	221112233	ul. Mleczna 5, Warszawa
4		Hurtownia Warzyw Świeżość	1112223344	583337788	None
5		Piekarnia Chrupiący Róg	9876543210	223334455	ul. Żytnia 12, Kraków
6		Źródło Natury Sp. z o.o.	3334445566	712223344	ul. Zdrojowa 1, Wrocław
7		Pupil Food Sp. z o.o.	5556667788	413334455	None
8		Mrożny Świat S.A.	7776665544	125551122	ul. Lodowa 44, Łódź
9		Sady Mazowieckie	9998887766	254449900	ul. Owocowa 3, 05-600 Grójec
10		Młyn i Spółka	4445556677	174445566	ul. Pszenna 2, Rzeszów
11		Herbaciarnia Świata	6667778899	855556677	ul. Aromatyczna 9, Białystok
12		Przyprawy Orientu	3332221100	814445566	None
13		Czysty Dom Hurt	8889990011	521112233	ul. Chemiczna 30, Bydgoszcz
14		Techno-Sklep Hurtownia	7770001112	565556677	ul. Przemysłowa 50, 87-100 Toruń
15		Piękno i Zdrowie	1110002223	342223344	ul. Urody 4, Kalisz

KLIENCI

System Zarządzania Sklepem

KATEGORIE DOSTAWCY **KLIENCI** MAGAZYNY PRODUKTY RAPORTY

Wyszukiwanie (Kryteria)

Kolumna: IMIE Fraz: Szukaj Resetuj

Odśwież Dodaj nowy Usuń zaznaczone

ID_KLIENTA	IMIE	NAZWISKO	NR_TEL	EMAIL	KARTA_RABATOWA
1	Jan	Kowalski	601234567	j.kowalski@poczta.pl	KRT-10001
2	Anna	Nowak	784112233	a.nowak@poczta.pl	None
3	Piotr	Wiśniewski	512998776	p.wisniewski@gmail.com	KRT-10002
4	Maria	Wójcik	660554332	m.wojcik@wp.pl	None
5	Krzysztof	Lewandowski	None	k.lewandowski@poczta.pl	KRT-10003
6	Agnieszka	Zielińska	881223344	a.zielinska@gmail.com	KRT-10004
7	Tomasz	Szymański	505443221	t.szymanski@poczta.pl	None
8	Małgorzata	Woźniak	692887112	m.wozniak@wp.pl	KRT-10005
9	Paweł	Kozłowski	730111999	p.kozlowski@gmail.com	None
10	Barbara	Jankowska	531442553	b.jankowska@poczta.pl	KRT-10006
11	Marcin	Mazur	609887776	m.mazur@wp.pl	KRT-10007
12	Elżbieta	Krawczyk	888777666	e.krawczyk@gmail.com	None
13	Michał	Kaczmarek	501555666	m.kaczmarek@poczta.pl	KRT-10008
14	Zofia	Stępień	None	None	None
15	Adam	Grabowski	722333444	a.grabowski@gmail.com	KRT-10009
16	Jolanta	Kwiatkowska	604506708	None	None
17	Robert	Kamiński	None	r.kaminski@wp.pl	None

MAGAZYNY

System Zarządzania Sklepem

KATEGORIE DOSTAWCY **KLIENCI** **MAGAZYNY** PRODUKTY RAPORTY

Wyszukiwanie (Kryteria)

Kolumna: SEGMENT Fraz: Szukaj Resetuj

Odśwież Dodaj nowy Usuń zaznaczone

ID_MAGAZYNU	SEGMENT	OPIS
1	A-01	Chłodnia: produkty nabiałowe i sery
2	A-02	Chłodnia: mięso, wędliny i drób
3	B-01	Magazyn suchy: kasze, mąki, makarony
4	B-02	Magazyn suchy: słodczyce i przekąski
5	C-01	Strefa napojów: wody i soki
6	C-02	Strefa napojów: napoje gazowane i energetyki
7	D-01	Dział warzywny: warzywa korzeniowe
8	D-02	Dział owocowy: owoce egzotyczne
9	E-01	Mroźnie: lody i owoce mrożone
10	E-02	Mroźnie: gotowe dania i warzywa mrożone
11	F-01	Chemia domowa: środki czystości
12	F-02	Kosmetyki i higiena osobista
13	G-01	Karma i akcesoria dla zwierząt
14	H-01	Artykuły przemysłowe i sezonowe
15	I-01	Strefa zwrotów i reklamacji

PRODUKTY

System Zarządzania Sklepem

KATEGORIE

DOSTAWCY

KLIENCI

MAGAZYNY

PRODUKTY

RAPORTY

Wyszukiwanie (Kryteria)

Kolumna: NAZWA

Fraza:

Szukaj

Resetuj

Odśwież

Dodaj nowy

Usuń zaznaczone

ID_PRODUKTU	NAZWA	CENA	JEDNOSTKA_MIARY	KOD_KRESKOWY	DOSTAWCA	MAGAZYN	KATEGORIA	ALERGENY
1	Mleko 2% UHT	3.49	szt	5901234567890	Polmlek Sp. z o.o.	A-01	Nabiał i jaja	Laktoza
2	Szynka Konserwowa	42.5	kg	2100550000001	Zakłady Mięsne Dobropol	A-02	Mięso, drób i wędliny	Białko sojowe, Gorczyca
3	Chleb Żytni	4.2	szt	None	Piekarnia Chrupiący Róg	B-01	Pieczynwo i wyroby cuk	Gluten
4	Pomidory Malinowe	14.99	kg	None	Hurtownia Warzyw Świeżość	D-01	Warzywa świeże	None
5	Jabłka Ligol	3.8	kg	None	Sady Mazowieckie	D-02	Owoce świeże	None
6	Lody Waniliowe 1L	18.5	szt	5904433221100	Mrożny Świat S.A.	E-01	Mrożonki i dania gotowe	Jaja, Laktoza
7	Woda Gazowana 1.5L	2.2	szt	5905566778899	Źródło Natury Sp. z o.o.	C-01	Napoje, wody i soki	None
8	Czekolada Mleczna	4.8	szt	7622210001234	Słodka Dolina	B-02	Słodycze i słone przekąski	Laktoza, Lecytyna sojowa
9	Mąka Pszenna 1kg	3.1	szt	5908877665544	Młyn i Spółka	B-01	Produkty sypkie (mąka)	Gluten
10	Kawa Ziarnista 500g	45.0	szt	4001234567890	Herbaciarnia Świata	C-02	Kawy, herbaty i kakao	None
11	Płyn do Naczyń	8.9	szt	5412345678901	Czysty Dom Hurt	F-01	Chemia domowa i środki czyszczące	None
12	Krem do rąk	12.5	szt	4005800123456	Piękno i Zdrowie	F-02	Kosmetyki i higiena osobista	None
13	Karma dla psa 5kg	35.0	szt	5903322114455	Pupil Food Sp. z o.o.	G-01	Karma i akcesoria dla zwierząt	Zboża
14	Pieprz Czarny	1.5	szt	5900000001112	Przyprawy Orientu	H-01	Przyprawy i dodatki do żywności	None
15	Baterie AA (4szt)	15.99	szt	4008400123456	Techno-Sklep Hurtownia	H-01	Artykuły przemysłowe	None
16	Jogurt Naturalny 400g	2.8	szt	5901122334455	Polmlek Sp. z o.o.	A-01	Nabiał i jaja	Laktoza
17	Bułka Kajzerka	0.85	szt	None	Piekarnia Chrupiący Róg	B-01	Pieczynwo i wyroby cuk	Gluten
18	Kurczak Cały	18.9	kg	2100880000002	Zakłady Mięsne Dobropol	A-02	Mięso, drób i wędliny	Gorczyca
19	Marchew Polska	3.5	kg	None	Hurtownia Warzyw Świeżość	D-01	Warzywa świeże	None
20	Banany Premium	6.99	kg	None	Sady Mazowieckie	D-02	Owoce świeże	None
21	Pizza Mrożona 4 Sery	12.5	szt	5909988776655	Mrożny Świat S.A.	E-02	Mrożonki i dania gotowe	Gluten, Laktoza
22	Sok Pomarańczowy 1L	5.8	szt	5907766554433	Źródło Natury Sp. z o.o.	C-01	Napoje, wody i soki	None

RAPORTY

System Zarządzania Sklepem

KATEGORIE

DOSTAWCY

KLIENCI

MAGAZYNY

PRODUKTY

RAPORTY

Raport Produktów

Cena min: 0

Cena max: 10

Generuj

Statystyka Magazynu

Generuj Raport z Wykresem

Karta Klienta

Nazwisko klienta:

Generuj Kartę

Lista Dostawców

Generuj Listę